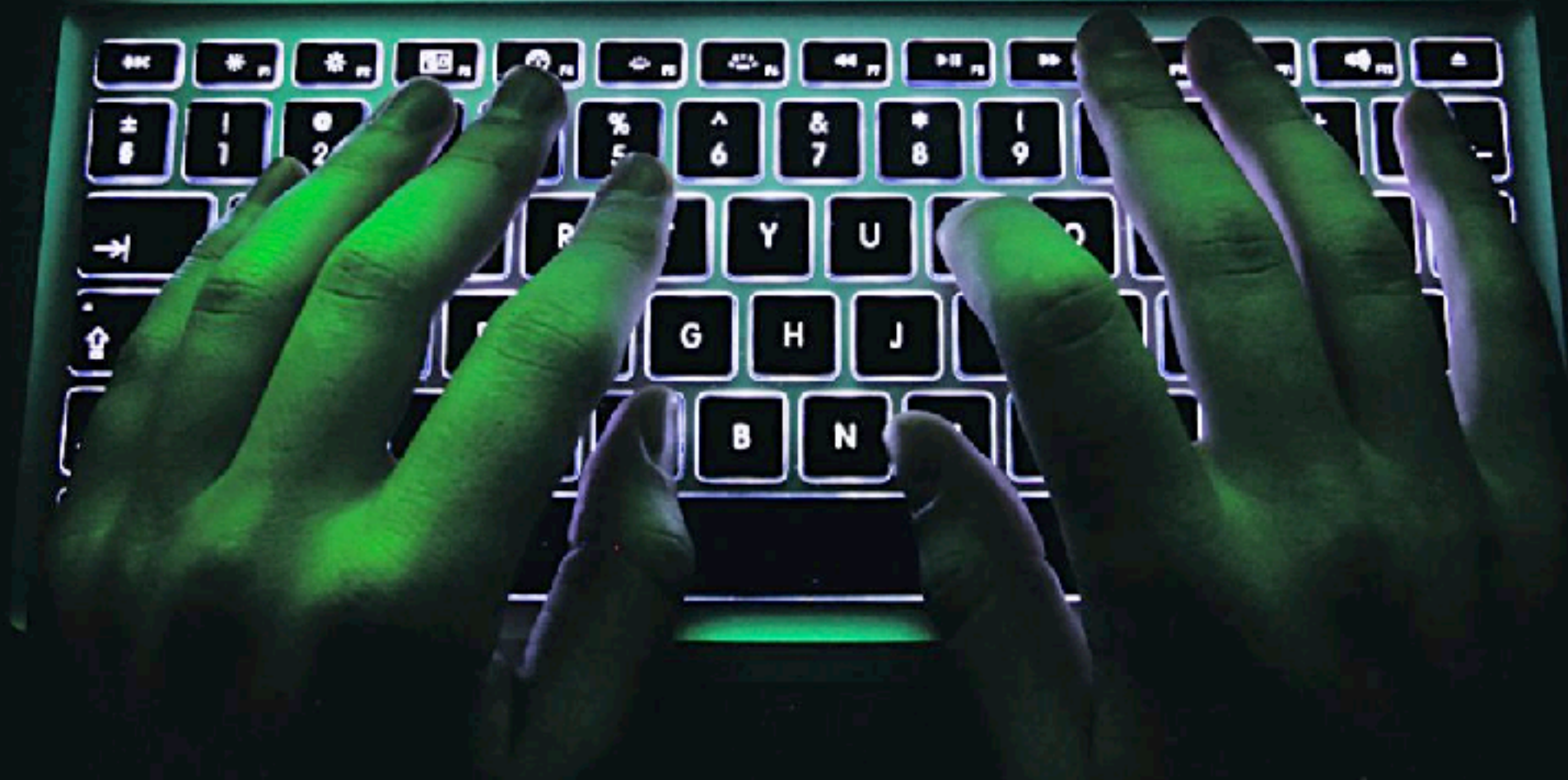


PROGRAMMIERUNG 2

Kapitel 1: **Einführung**



Prof. Dr. Markus Esch
htw saar

Rückblick

Programmierung 1



Kahoot!

go to
kahoot.it

PIN
eingeben



Nickname
eingeben



mitspielen



1

2

3

4





Programmier**paradigmen**

```
PROGRAM Add (input, output);
```

```
VAR number1, number2, total: integer;
```

```
PROCEDURE Obtain;
```

```
BEGIN
```

```
    WRITE('Please input the first number: ');
```

```
    READLN(number1);
```

```
    WRITE('Please input the second number: ');
```

```
    READLN(number2);
```

```
END;
```

```
PROCEDURE Calculate;
```

```
BEGIN
```

```
    total := number1 + number2;
```

```
END;
```

```
PROCEDURE Display;
```

```
BEGIN
```

```
    WRITELN('The total is:', total);
```

```
END;
```

```
BEGIN
```

```
    obtain;
```

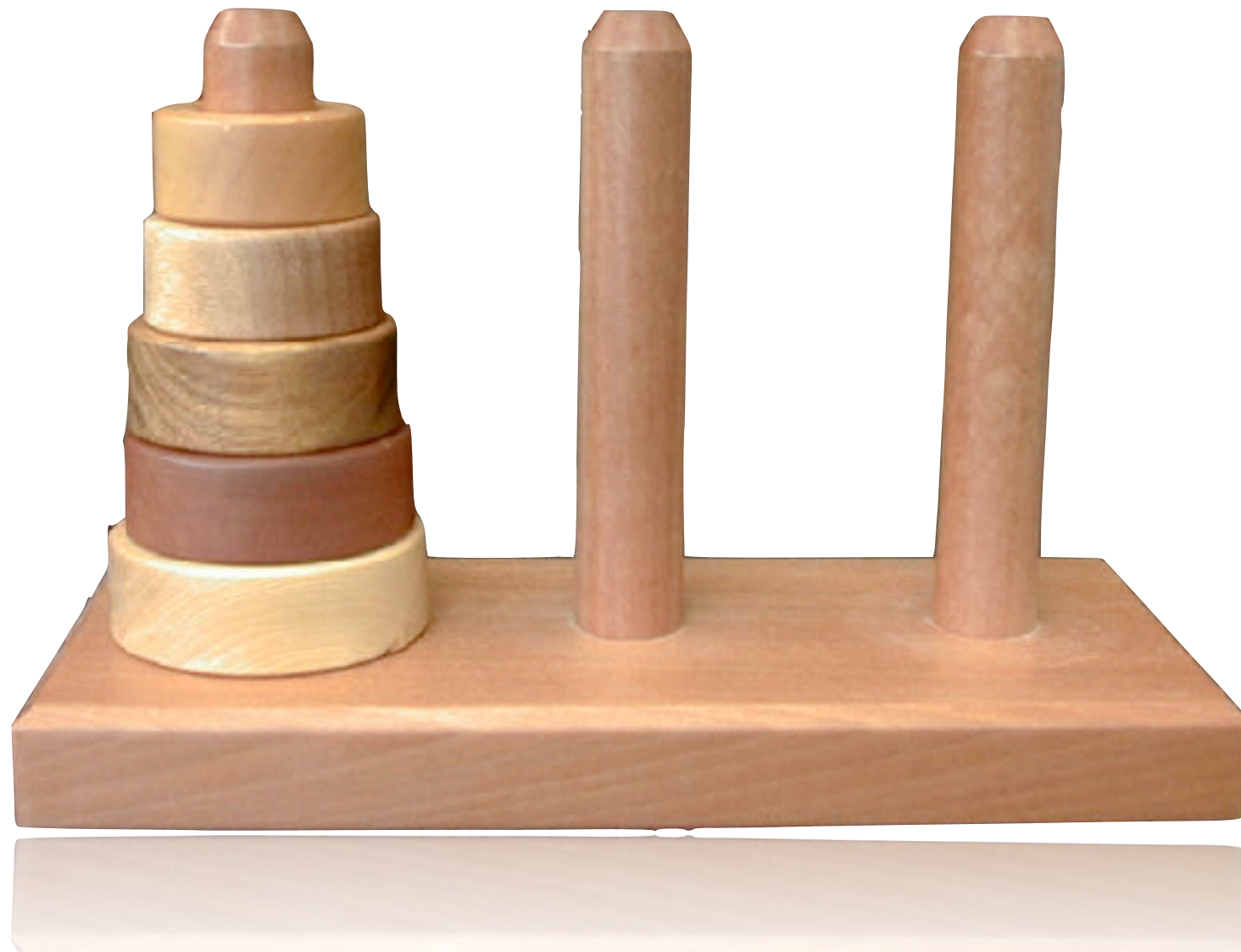
```
    calculate;
```

```
    display;
```

```
END.
```

Imperativ: Sequenz
von Anweisungen

```
(defun toh (height start-peg-name extra-peg-name end-peg-name)
  (cond ((= height 0) nil)
        ((= height 1) (list (cons start-peg-name end-peg-name)))
        (t (append (toh (1- height) start-peg-name end-peg-name extra-peg-name)
                    (toh 1 start-peg-name extra-peg-name end-peg-name)
                    (toh (1- height) extra-peg-name start-peg-name end-peg-name)))))
```



Funktionale Programmierung

Fakten

```
father(sam, john).  
father(john, jimmy).
```

Regeln

```
grandpa(X, Y) :- father(X, Z), father(Z, Y)
```

Fragen

```
[user]    ?- grandpa(sam, jimmy)
```

```
[system]  yes
```

```
[user]    ?- father(X, jimmy).
```

```
[system]  X = john
```

Logische Programming

```

public class Car {
    private String model;
    private int speed;

    public Car(String model) {
        this.model = model;
        this.speed = 0; // Initial speed is 0
    }

    public void drive(int increaseSpeed) {
        speed += increaseSpeed;
        System.out.println(model + " is now driving at " + speed + "
km/h.");
    }

    public void brake(int decreaseSpeed) {
        speed -= decreaseSpeed;
        if (speed < 0) {
            speed = 0;
        }
        System.out.println(model +
        " slows down to " + speed + " km/h.");
    }
}

```

Objektorientierte Programmierung

Warum Objektorientierung?



Vorteile der Objektorientierung



1 Minute

Vorteile der Objektorientierung

Ausdruckskraft

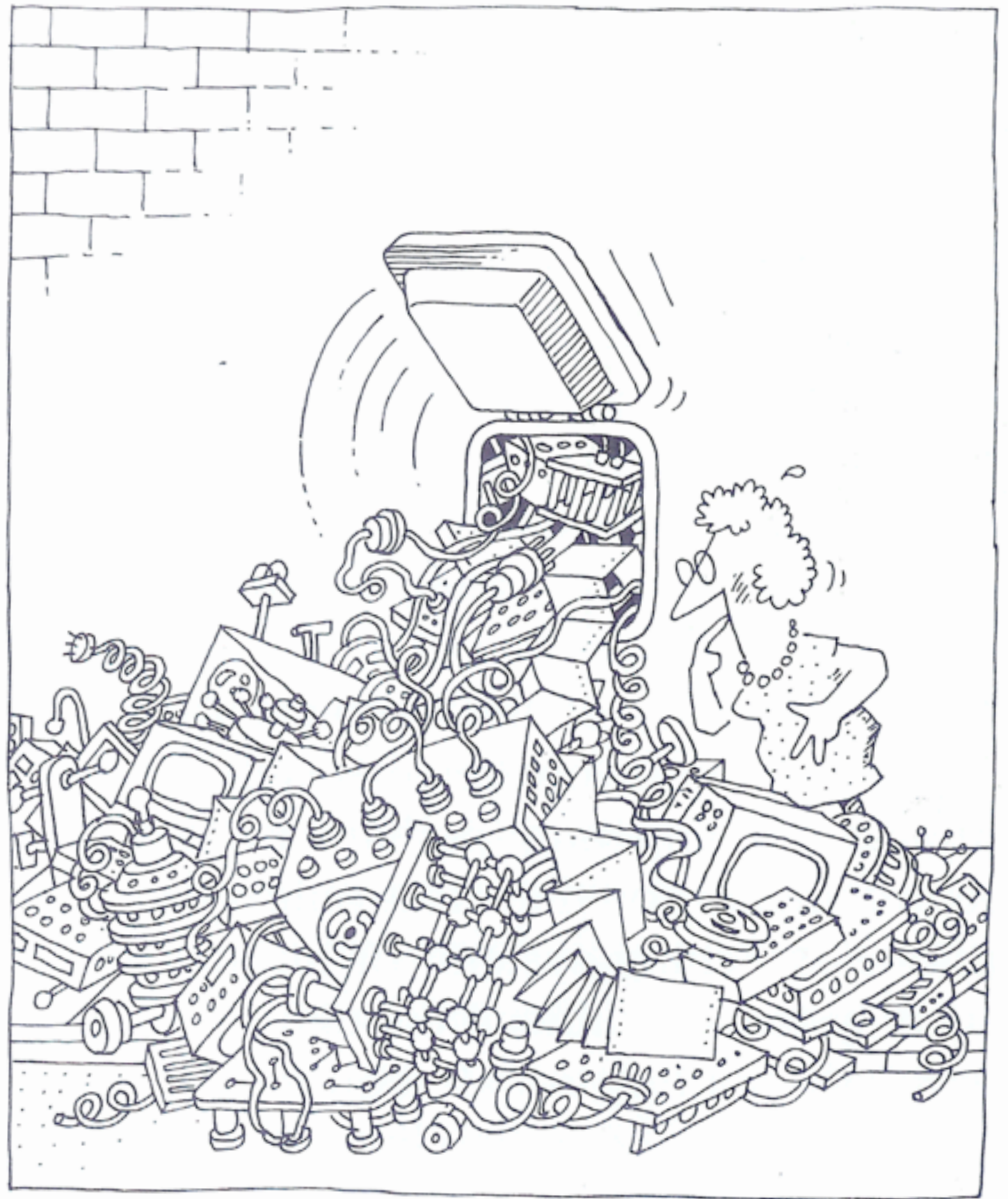
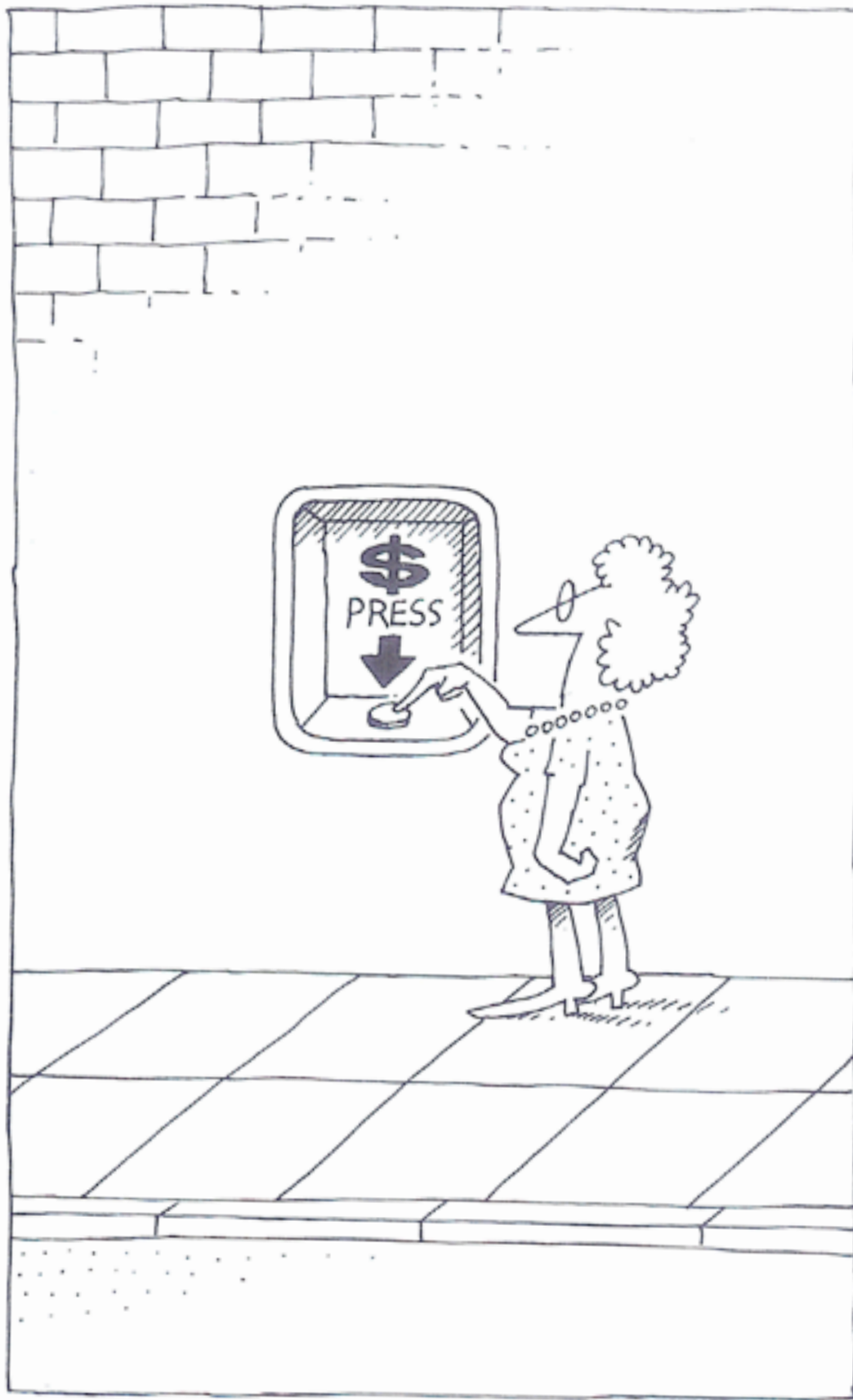
Resilienz

Wartbarkeit

Modularität

Wiederverwendung

Erweiterbarkeit

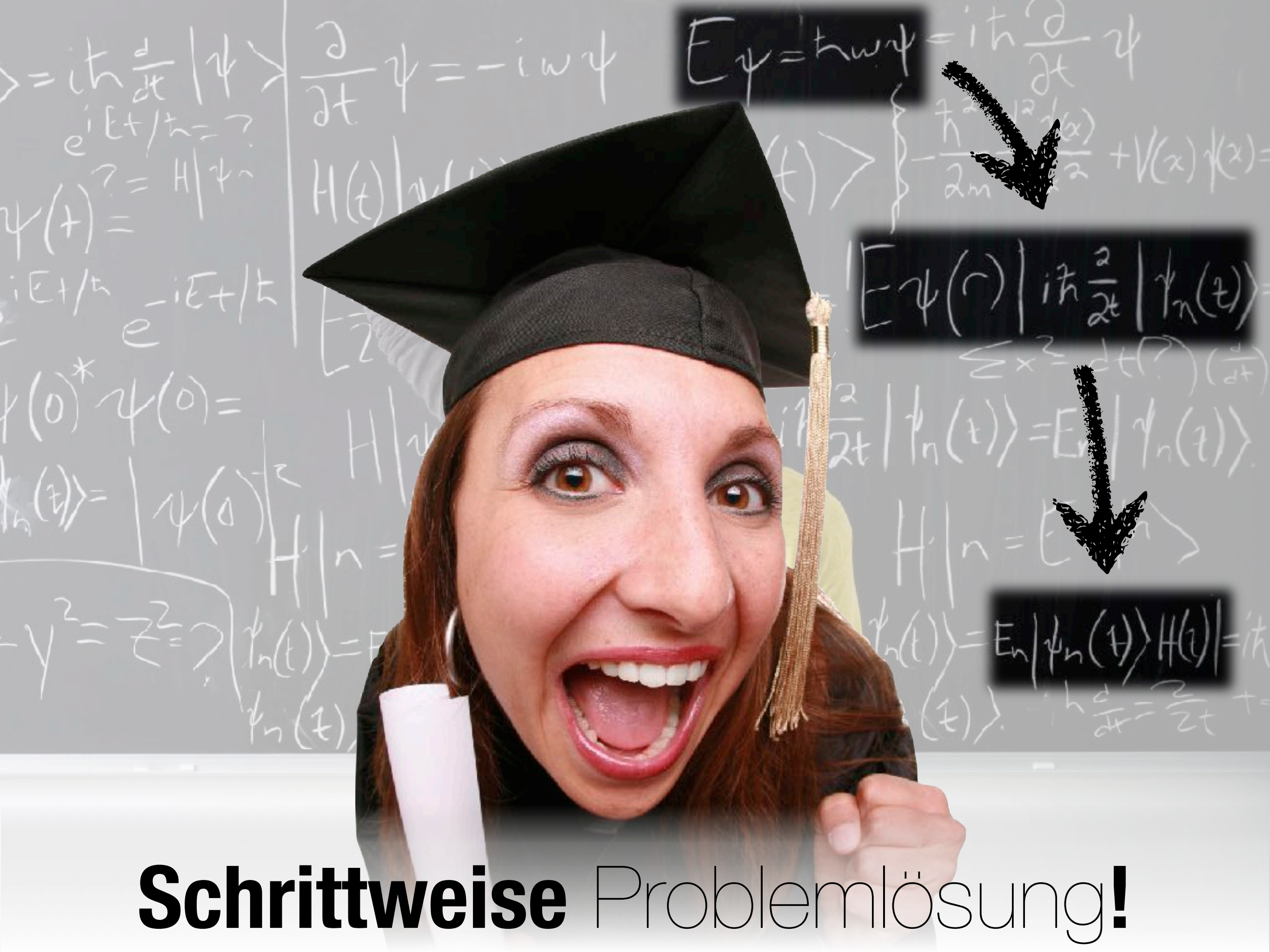


Software **Komplexität**

$\psi = i\hbar \frac{d}{dt} |\psi\rangle$
 $e^{iEt/\hbar} = ?$
 $? = H|\psi\rangle$
 $\psi(t) =$
 $iEt/\hbar \quad -iEt/\hbar$
 $\psi(0)^* \psi(0) =$
 $\psi(t) = |\psi(0)|^2$
 $H|n\rangle = E_n|n\rangle$
 $\psi(t) = E_n|\psi(t)\rangle H(t) = i\hbar$
 $\psi(t) = i\hbar \frac{d}{dt} = \frac{2}{2t} +$

$\frac{\partial}{\partial t} \psi = -i\omega \psi$
 $E\psi = \hbar\omega\psi = i\hbar \frac{\partial}{\partial t} \psi$
 $H(t)|\psi(t)\rangle =$
 $\left\{ -\frac{\hbar^2}{2m} \frac{d^2 \psi(x)}{dx^2} + V(x)\psi(x) \right\}$
 $[E - \frac{\hbar^2}{2m} \frac{d^2}{dx^2} + U(x)]\psi(x) = [E\psi(x) | i\hbar \frac{d^2}{dt} |\psi_n(t)\rangle =$
 $\sum x^2 = dt(?) (\frac{d}{dt})$
 $i\hbar \frac{d^2}{dt} |\psi_n(t)\rangle = E_n |\psi_n(t)\rangle$
 $H|n\rangle = E_n|n\rangle$
 $\psi_n(t) = E_n|\psi_n(t)\rangle H(t) = i\hbar$
 $\psi_n(t) = i\hbar \frac{d}{dt} = \frac{2}{2t} +$

Vernetztes Denken ist **komplex**



Schrittweise Problemlösung!

Data

==

===

==

=====

=====

=====

Code

}}}

}}}

}}}

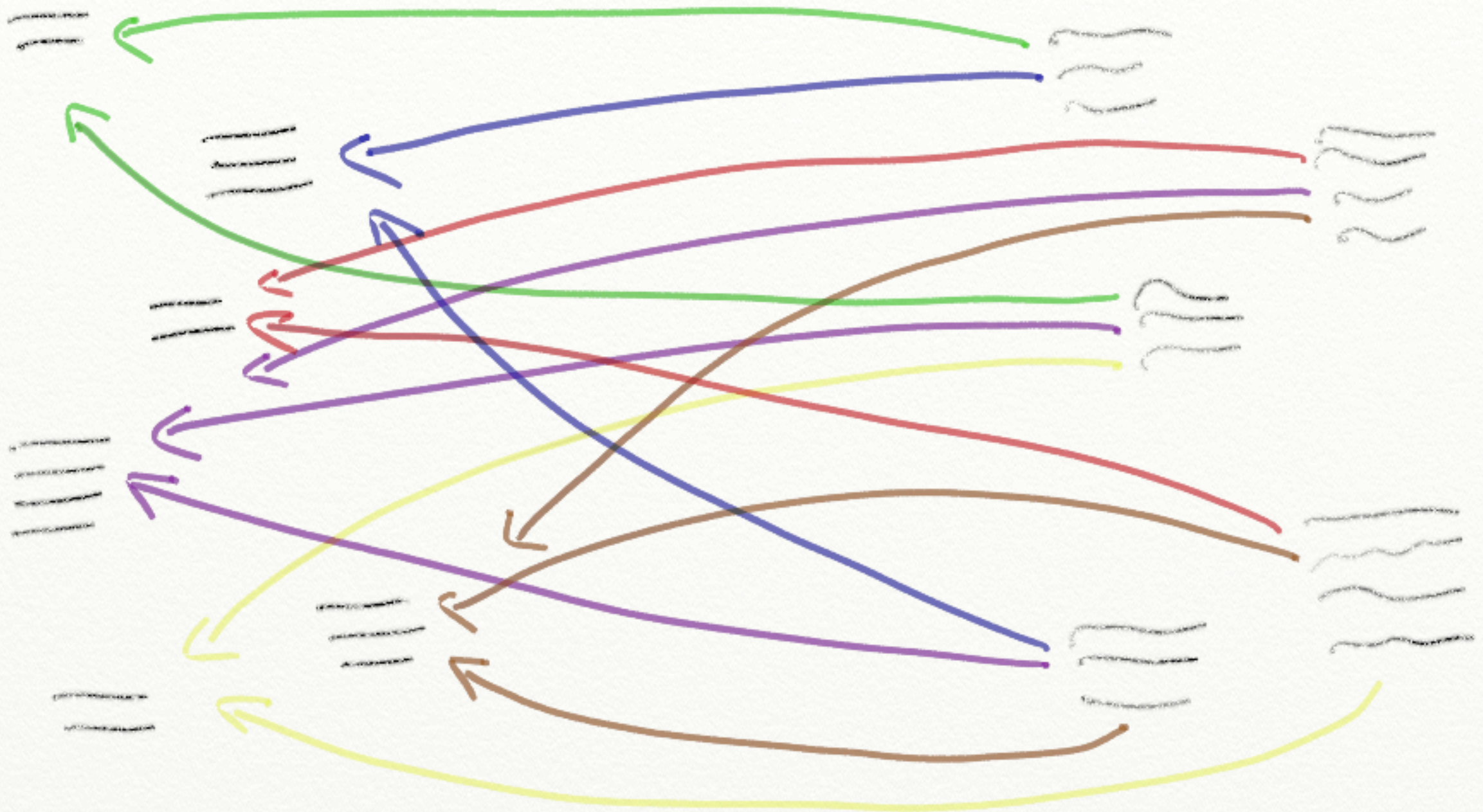
}}}

}}}

Imperative Programmierung

Data

Code



Unglaubliches **Durcheinander!**

Data

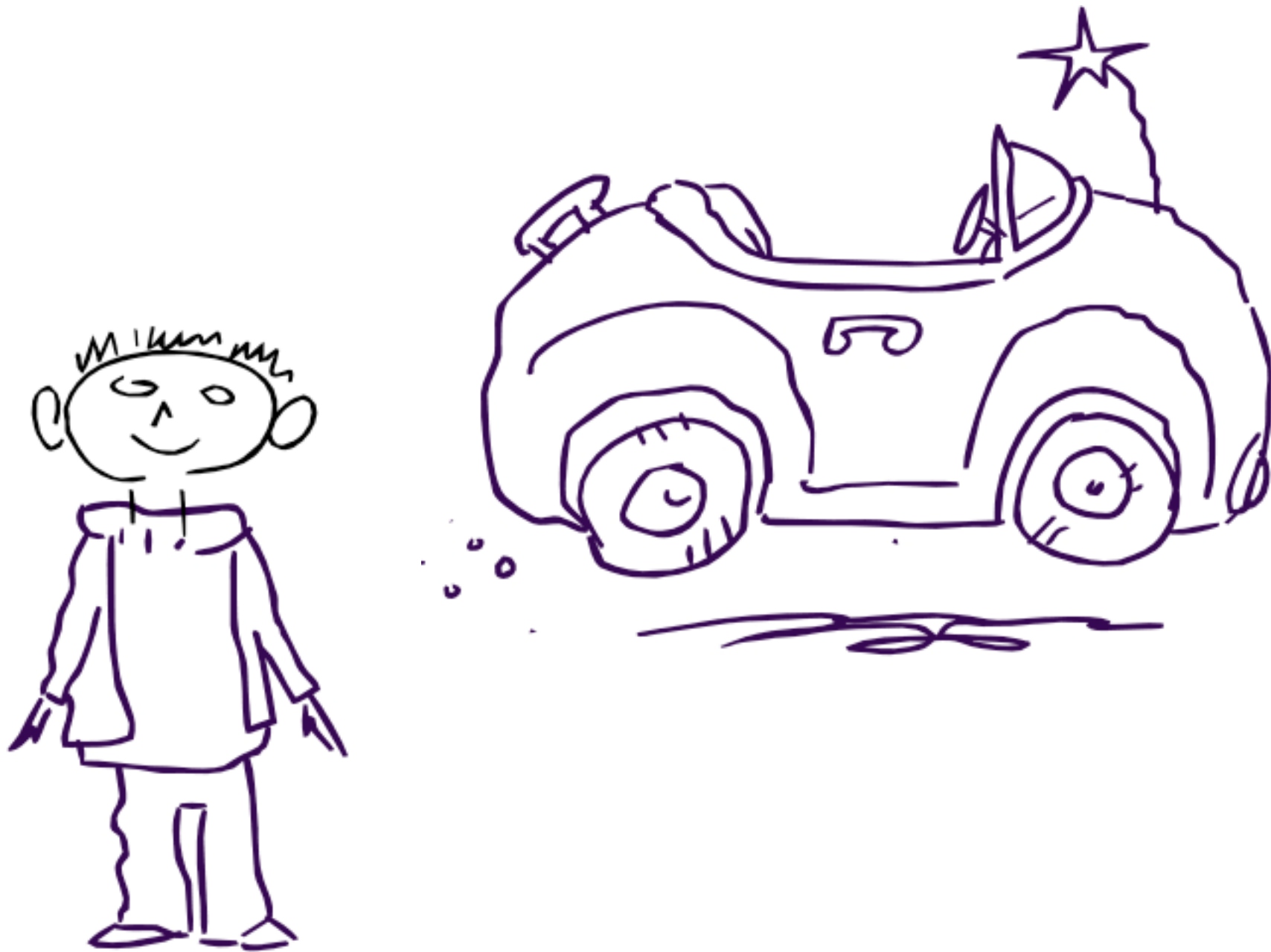
Code



Technozentrische Sicht



Anthropozentrische Sicht



Reale **Entitäten** und **Konzepte**



Person



Auto



Blume

Reale **Entitäten** und **Konzepte**



Properties

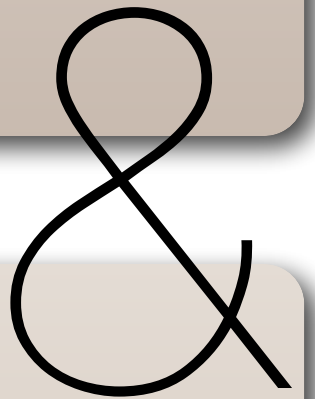
- ▶ manufacturer
- ▶ model
- ▶ horsepower
- ▶ fuelConsumption
- ▶ ...

Data

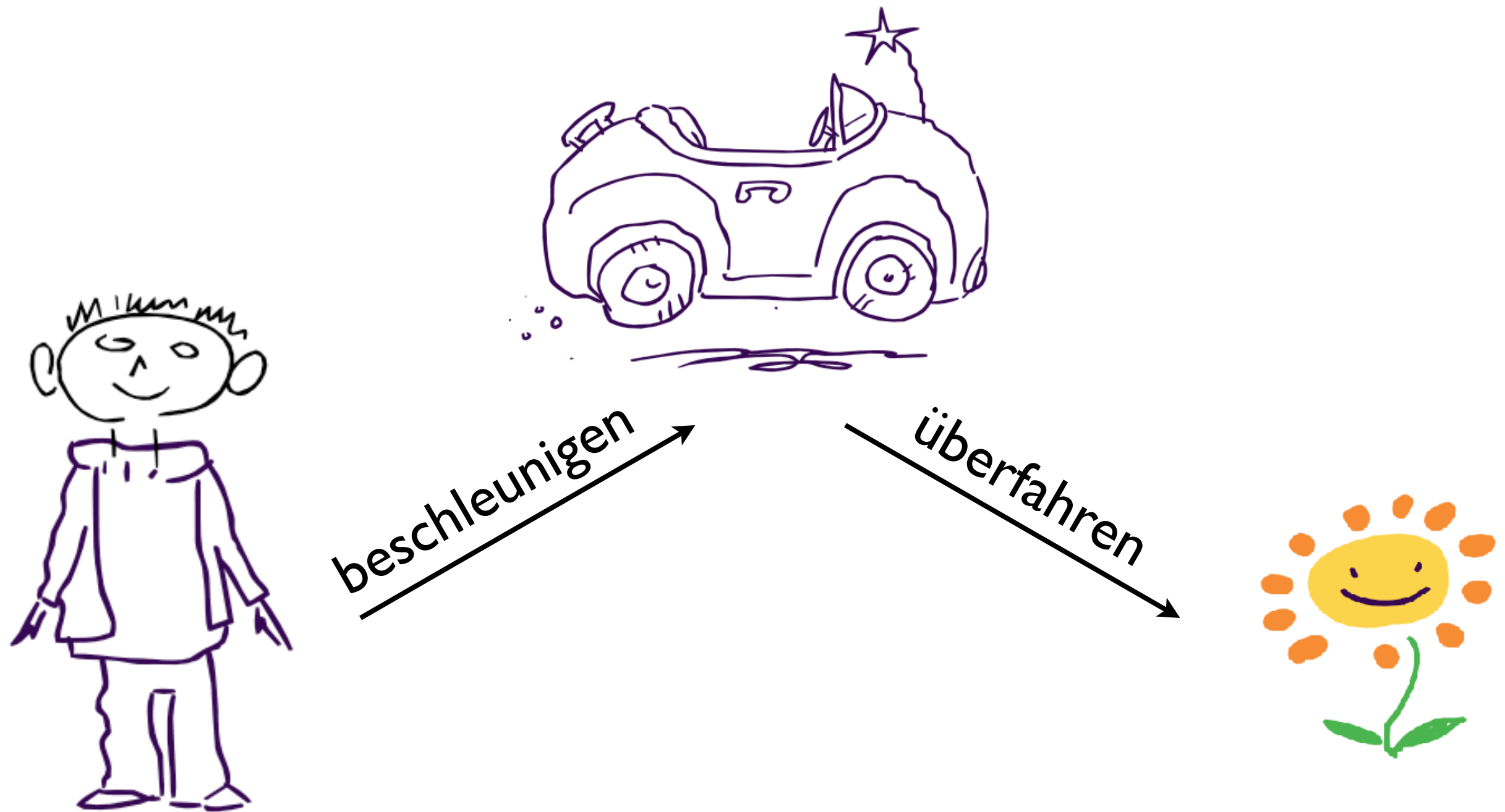
Behavior

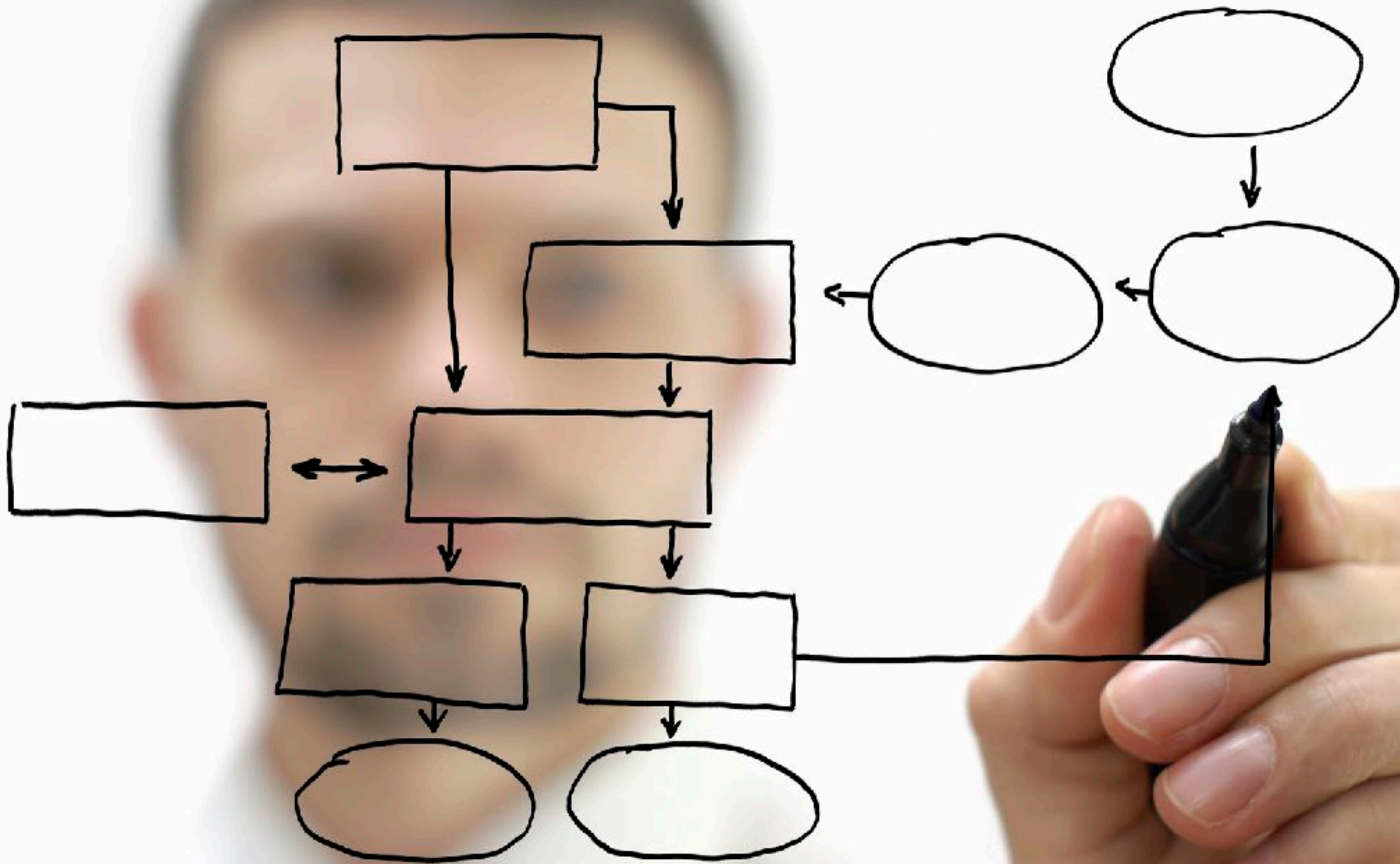
- ▶ startEngine()
- ▶ accelerate()
- ▶ brake()
- ▶ tootTooooooot()
- ▶ ...

Code



Natürliche **Interaktion**





Kommunizierende **Objekte**



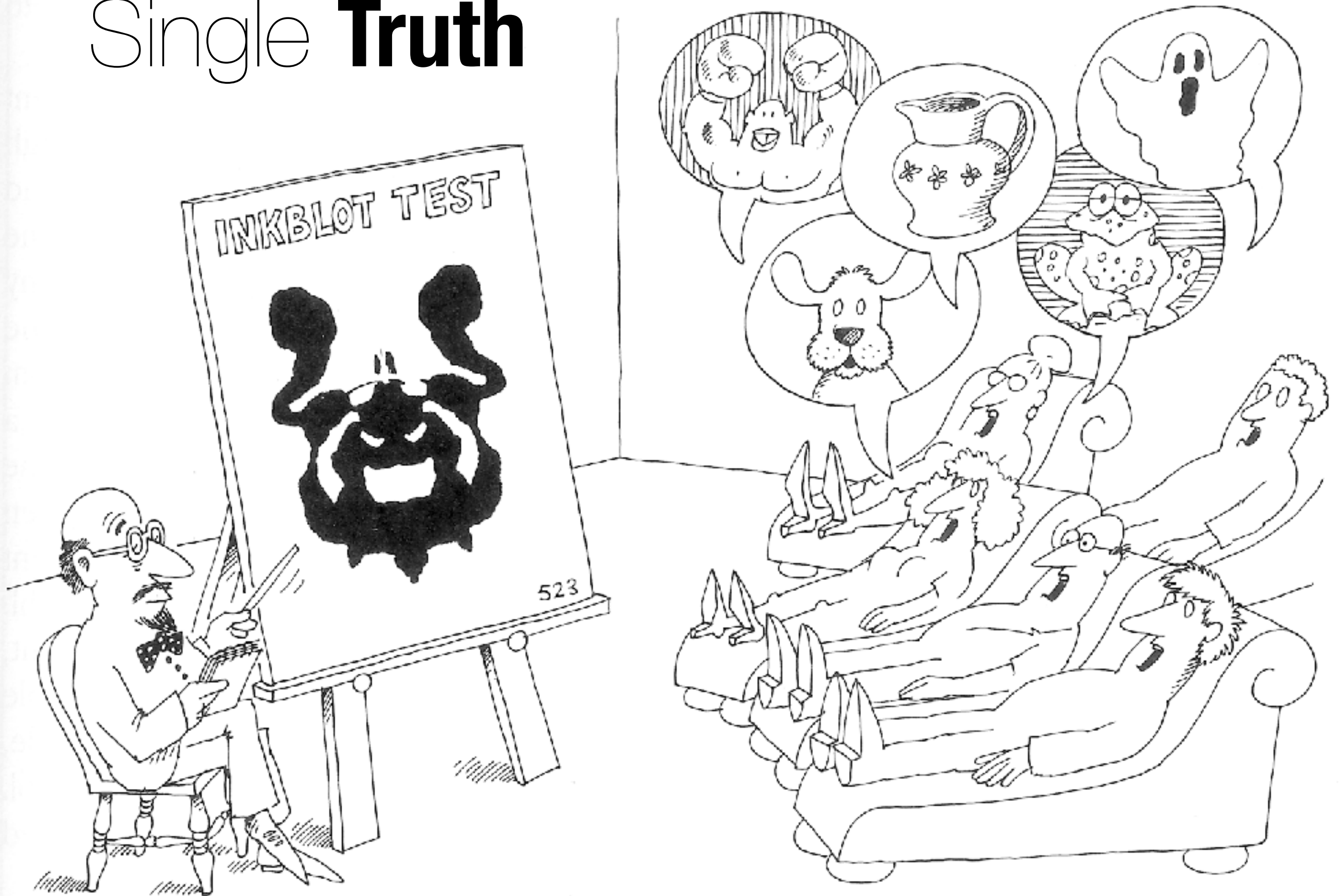
TRIZZI "A CENTRAL PARK FOR LOVERS" A/D '60...



Objektorientierte

Modellierung ist eine **Kunst**

There is **No**
Single **Truth**



Learning by Doing!



Object Oriented Programming in a **Nutshell**





Life in the **Library**



Which Objects ?
are there

Books



Readers



Librarian





Terrible **Oversimplification**



Properties



Behavior



Book



Properties

- ▶ title
- ▶ author
- ▶ publisher
- ▶ ISBN
- ▶ ...

Behavior

- ▶ Book(title, author, ...)
- ▶ getTitle()
- ▶ getAuthor()
- ▶ getPublisher()
- ▶ ...

Book

Reminder ...



Terrible **Oversimplification**

Rarely Pure Data-Containers

Add Useful **Behavior!**

Author:

Level of **Detail**

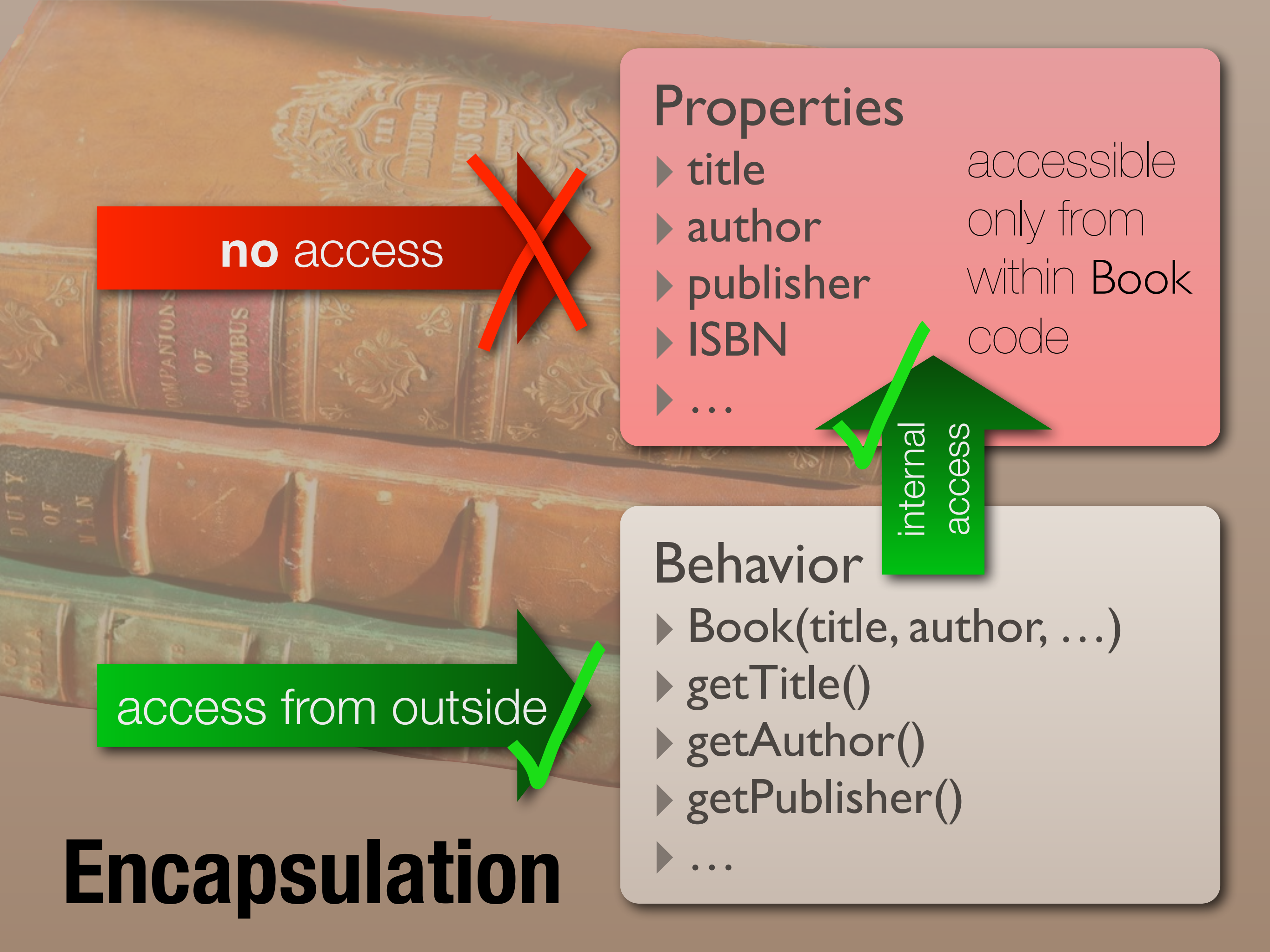
or just

William Shakespeare

William Shakespeare

- ▶ baptised April 26, 1564
- ▶ in Stratford-upon-Avon
- ▶ died April 23, 1616
- ▶ 38 plays, 154 sonnets, ...
- ▶ ...





no access

Properties

- ▶ title
 - ▶ author
 - ▶ publisher
 - ▶ ISBN
 - ▶ ...
- accessible
only from
within Book
code

internal
access

access from outside

Behavior

- ▶ Book(title, author, ...)
- ▶ getTitle()
- ▶ getAuthor()
- ▶ getPublisher()
- ▶ ...

Encapsulation



Class

&



Objects

```
class Book {
    private String title, author, publisher, isbn;

    public Book(String title, String author, String publisher,
                String isbn) { ... }

    public String getTitle() { return title; }
    public String getAuthor() { return author; }
    public String getPublisher() { return publisher; }
    public String getISBN() { return isbn; }
};
```

```
Book harry = new Book("Harry Potter Marries", "J.K. Rowling",  
                      "Bloomsbury Publishing", "978-0747595823");  
Book prince = new Book("Le Petit Prince", "A. de Saint-Exupéry",  
                       "Gallimard", "978-2070408504");
```



Reader

Properties

- ▶ name
- ▶ address
- ▶ readerNr
- ▶ ...

Behavior

- ▶ Reader(name, address, ...)
- ▶ getName()
- ▶ getAddress()
- ▶ ...



Reader

Properties

- ▶ name
- ▶ **address**
- ▶ readerNr
- ▶ ...

Behavior

- ▶ Reader(name, address, ...)
- ▶ getName()
- ▶ getAddress()
- ▶ ...

A Case for **Composition** . . .



Reader

- ▶ name
- ▶ address
- ▶ readerNr
- ▶ ...

Behavior

- ▶ Reader(name, address, ...)
- ▶ getName()
- ▶ getAddress()
- ▶ ...



Address re-use possible
in other Contexts



Address

Properties

- ▶ street
- ▶ number
- ▶ zipCode
- ▶ city
- ▶ ...

Behavior

- ▶ Address(street, ...)
- ▶ getStreet()
- ▶ getNumber()
- ▶ getZipCode()
- ▶ ...

Properties



Behavior



Librarian



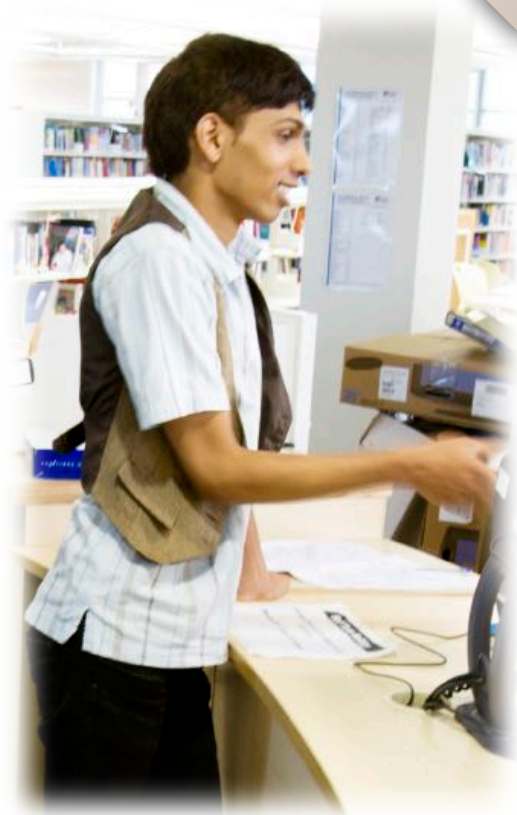
Reader

Librarian

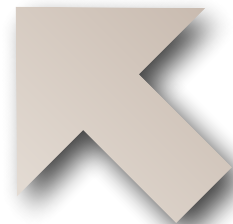
Inheritance



Person



Reader



Librarian

Inheritance of

Properties

&

Behaviour



Inheritance models

is-a

Relationship



Person



**Reader
is-a Person**



**Librarian
is-a Person**



Person



Reader

Properties

- ▶ name
- ▶ address
- ▶ readerNr
- ▶ ...

Behavior

- ▶ Reader(name, address, ...)
- ▶ getName()
- ▶ getAddress()
- ▶ ...



Librarian



Person



Reader

Properties

- ▶ name
- ▶ address
- ▶ readerNr
- ▶ ...

Behavior

- ▶ Reader(name, address, ...)
- ▶ getName()
- ▶ getAddress()
- ▶ ...



Librarian

Properties

- ▶ name
- ▶ address
- ▶ ...



Person

Behavior

- ▶ Person(name, address, ...)
- ▶ getName()
- ▶ getAddress()
- ▶ ...



Reader



Librarian

Properties

- ▶ name
- ▶ address
- ▶ ...



Person

Behavior

- ▶ Person(name, address, ...)
- ▶ getName()
- ▶ getAddress()
- ▶ ...



Reader

Properties

- ▶ readerNr
- ▶ ...

Behavior

- ▶ Reader(name, address, ...)
- ▶ ...



Librarian

Properties

- ▶ ...

Behavior

- ▶ ...

```
class Person {  
    private String name, address;  
  
    public Person(String name, String address) { ... }  
    // getters go here ...  
};
```

```
class Reader extends Person {  
    private int readerNr;  
  
    public Reader(String name, String address) {  
        super(name, address);  
        // create and set unique new readerNr ...  
    }  
};
```

Substitute

any **subclass**



for any of its **superclasses**



[Liskov Substitution Principle]

```
void meet(Person a, Person b) {  
    // talk to each other ...  
    // might use Person's interface only!  
}
```

```
Reader bonnie = new Reader(...);  
Reader clyde = new Reader(...);  
Librarian john = new Librarian(...);  
Book harry = new Book("Harry Potter Marries", "J.K. Rowling",  
    "Bloomsbury Publishing", "978-0747595823");
```

```
meet(bonnie, clyde);
```

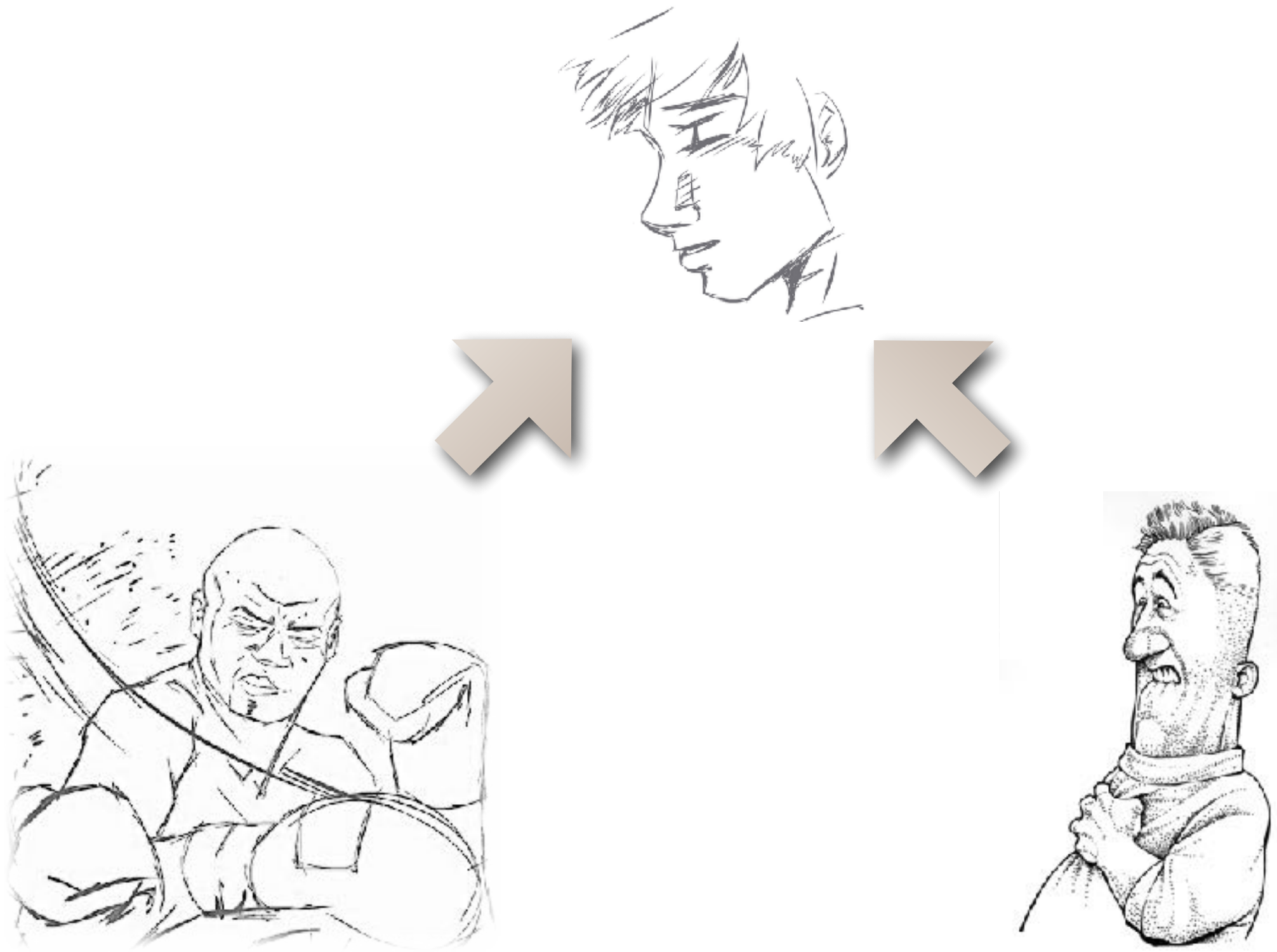


```
meet(clyde, john);
```



```
meet(john, harry);
```





Polymorphism — Changing Behavior

Example: **kickMe()**

Inheritance may

extend properties

change existing behavior

add new behavior

but **must not**

restrict existing properties
or behavior

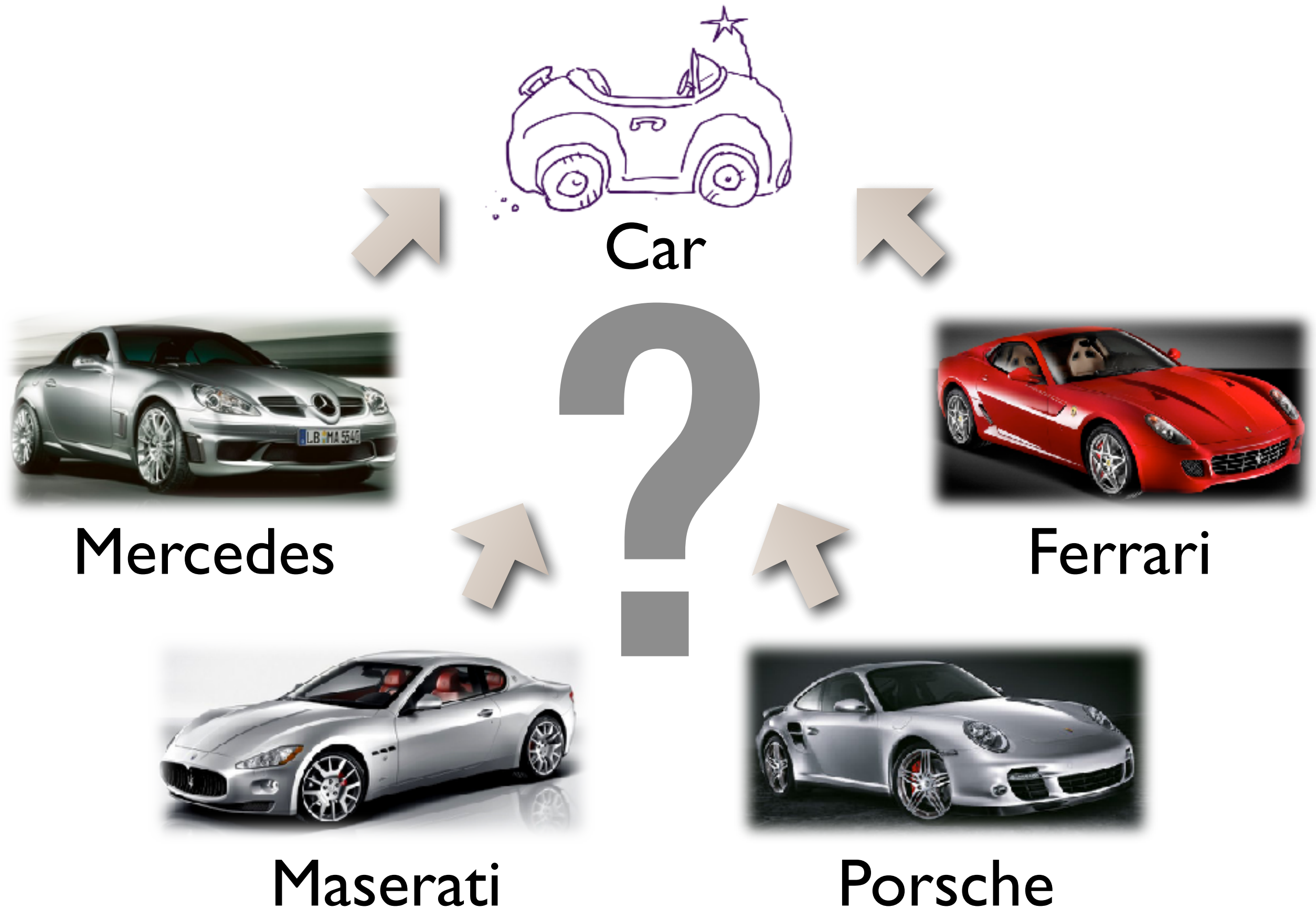


*If all you have is a
hammer, everything
looks like a nail.*



Use **Inheritance**
Judiciously!

Inheritance?



No! Just a Different Brand ...



Car



Mercedes



Ferrari



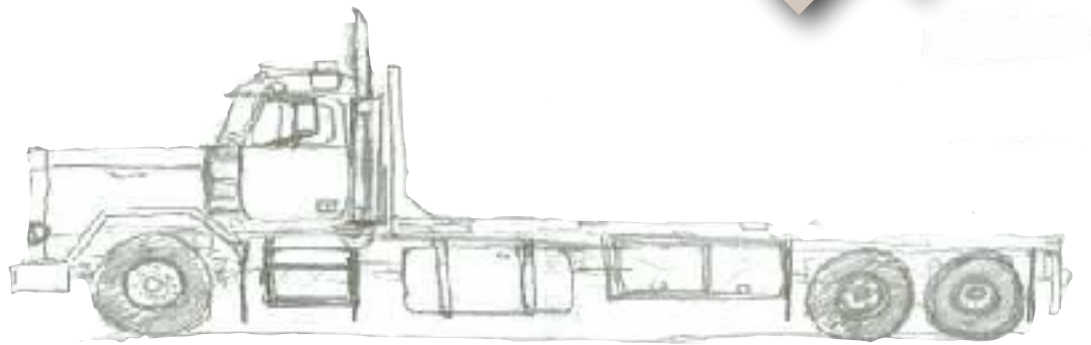
Maserati



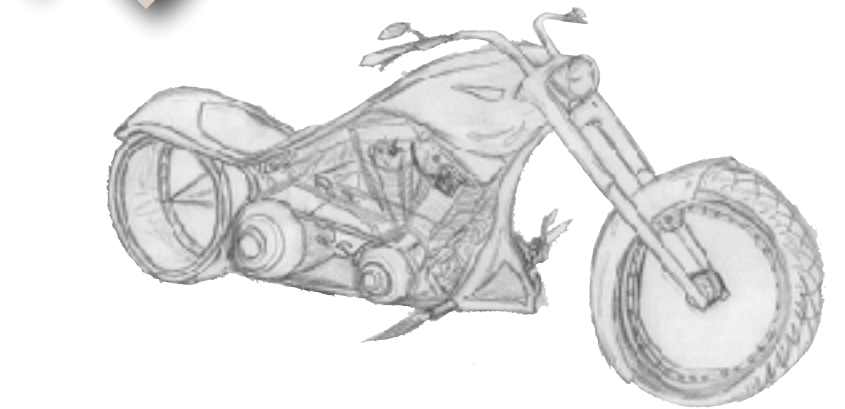
Porsche

Inheritance?

Vehicle



Truck



Motorcycle



Bicycle

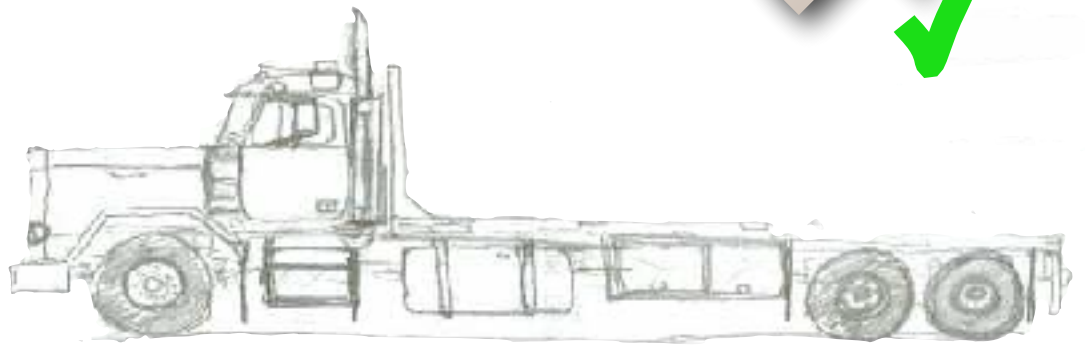


Car

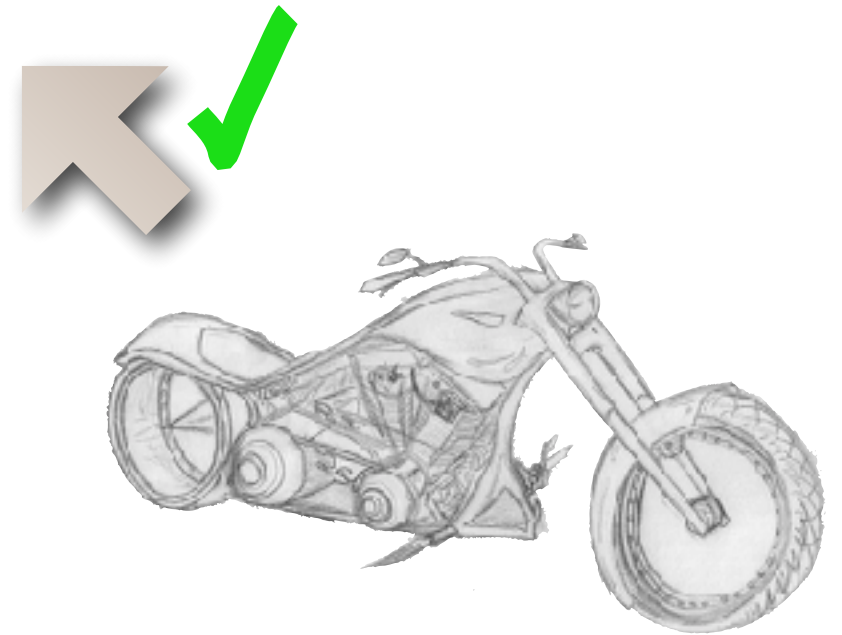


Inheritance!

Vehicle



Truck



Motorcycle



Bicycle



Car



Inheritance models



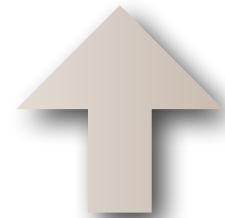
Relationship

closely examined ...

Quiz

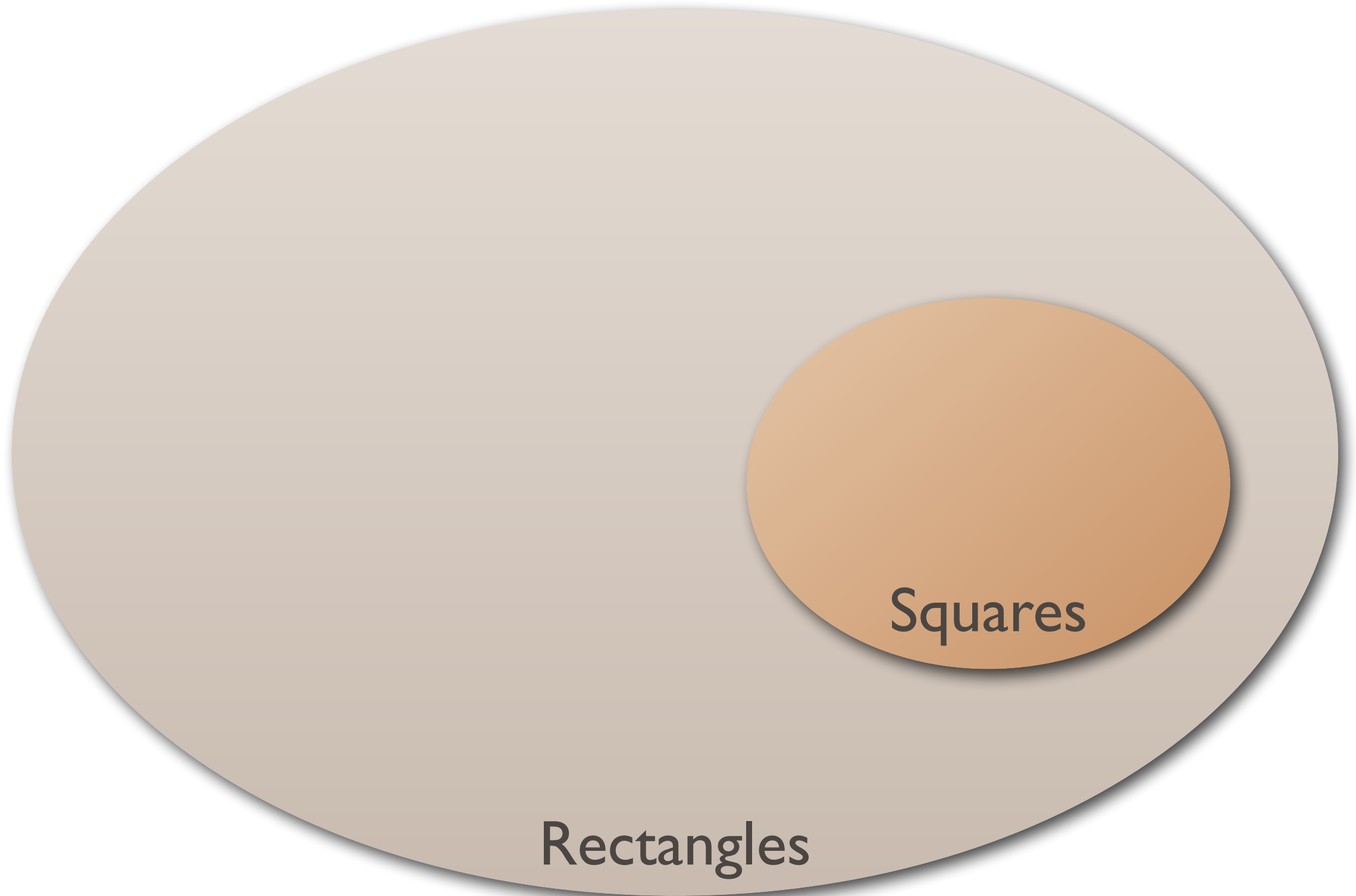


Square **is-a** Rectangle



Mathematics:

Squares $\subset$ Rectangles



but:

it's all about

behavior!

[Liskov Substitution Principle]

Rectangle

height : double
width : double

Rectangle(height : double, width : double)
getHeight() : double
getWidth() : double
setHeight(newHeight : double) : void
setWidth(newWidth : double) : void

← properties

← behavior

Rectangle
height : double width : double
Rectangle(height : double, width : double) getHeight() : double getWidth() : double setHeight(newHeight : double) : void setWidth(newWidth : double) : void

```
Rectangle r = new Rectangle(10, 20);
```

```
System.out.println(r.getHeight());    // prints 10
```

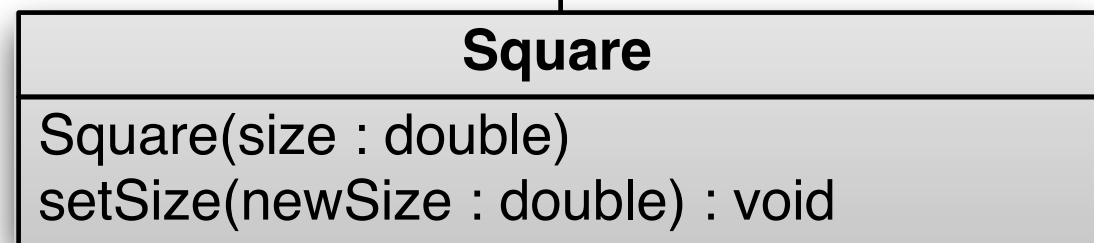
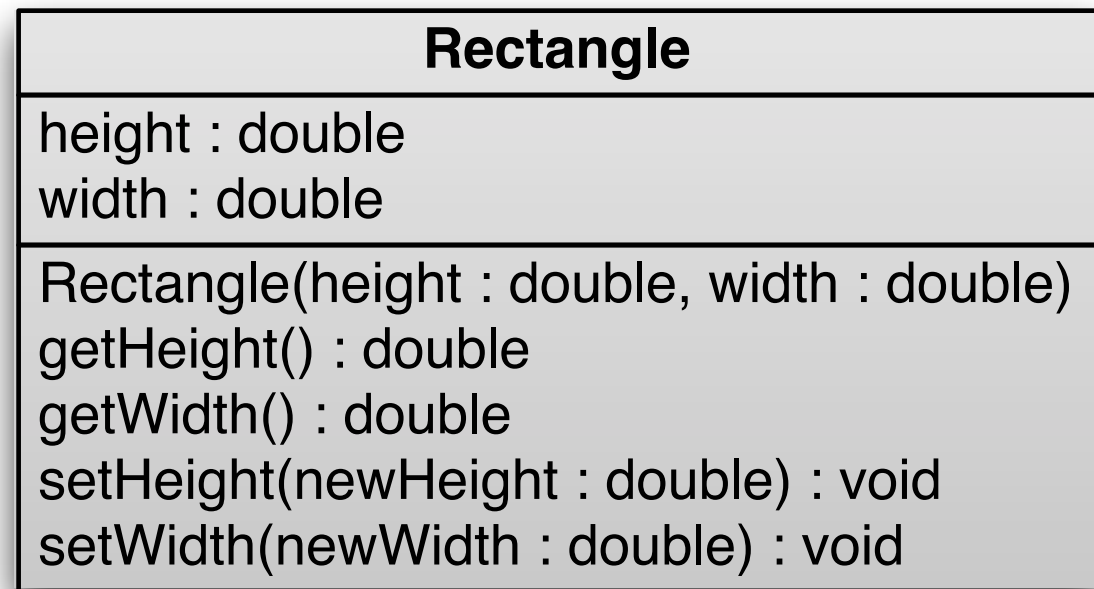
```
System.out.println(r.getWidth());     // prints 20
```

```
r.setHeight(111);
```

```
r.setWidth(222);
```

```
System.out.println(r.getHeight());    // prints 111
```

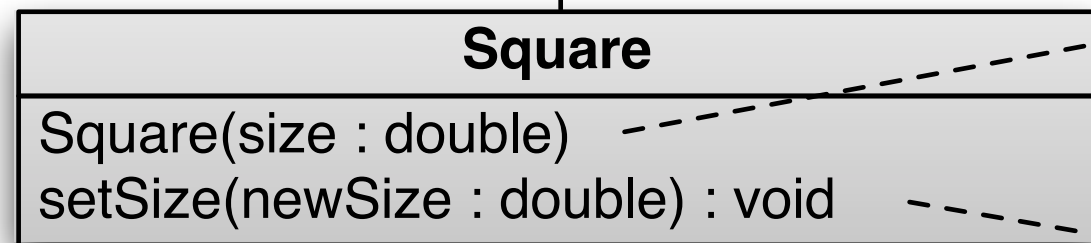
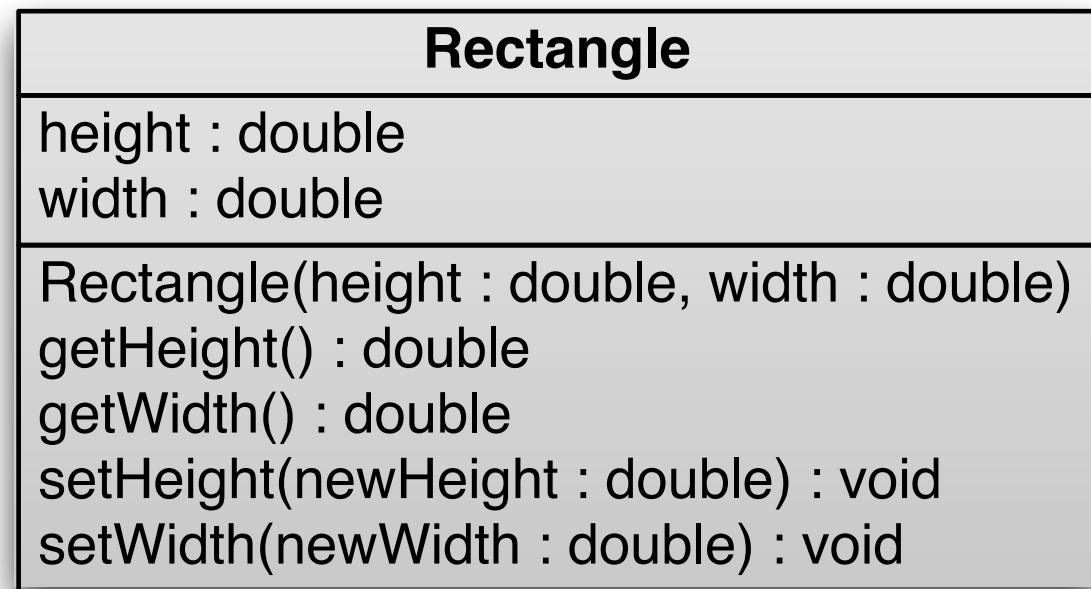
```
System.out.println(r.getWidth());     // prints 222
```



inheritance

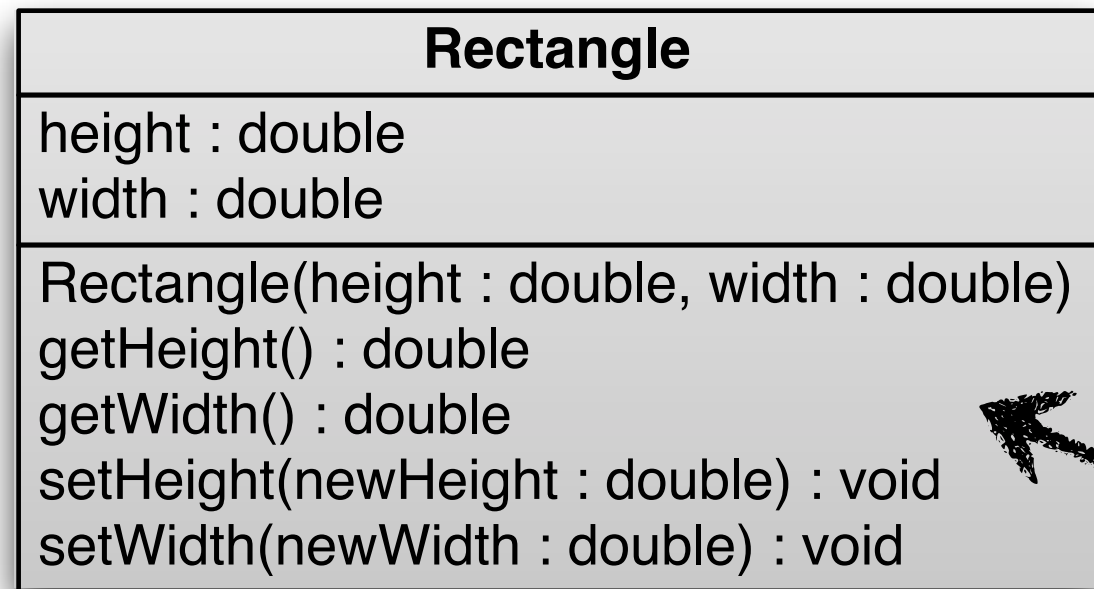


added behavior



```
Square(double size) {  
    super(size, size)  
}
```

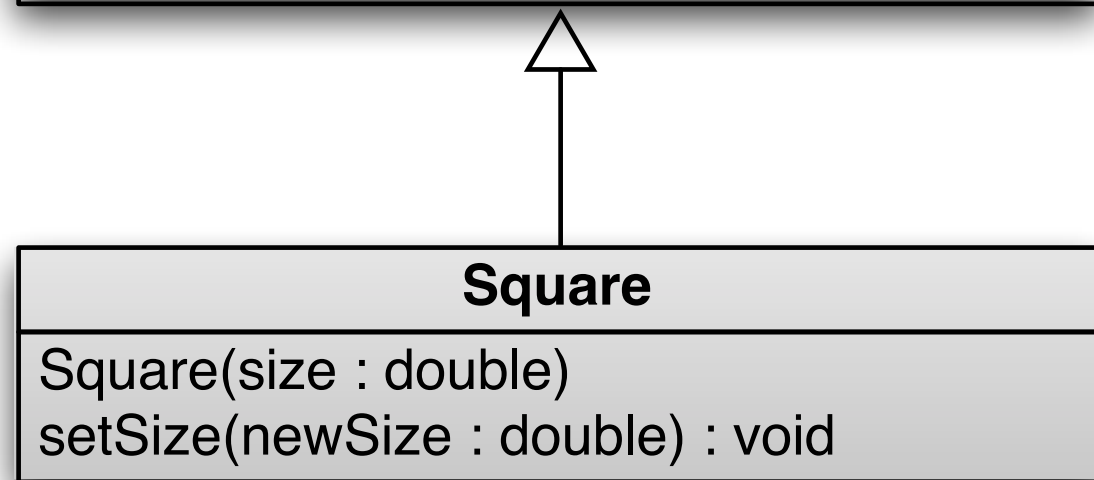
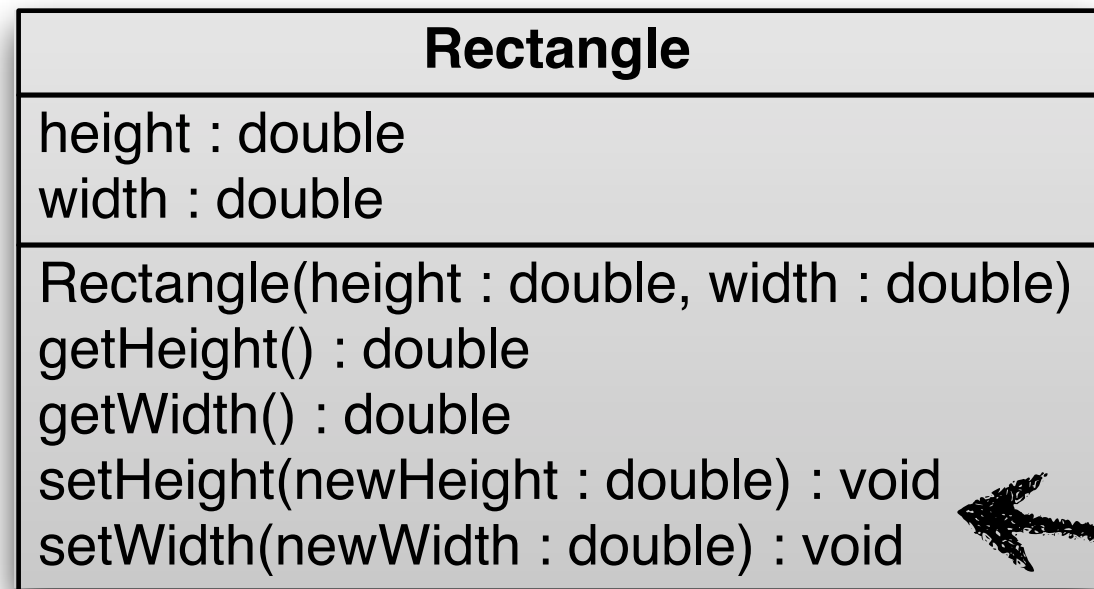
```
void setSize(double size) {  
    setHeight(size);  
    setWidth(size);  
}
```



getHeight() and
getWidth() are
inherited from
Rectangle

Square s = new Square(33);

System.out.println(s.getHeight()); // prints 33
System.out.println(s.getWidth()); // prints 33



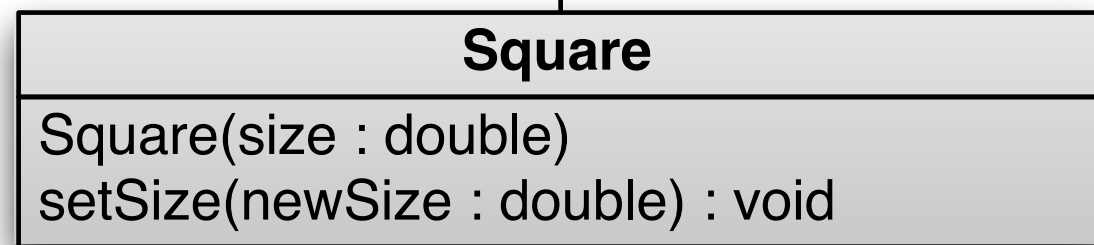
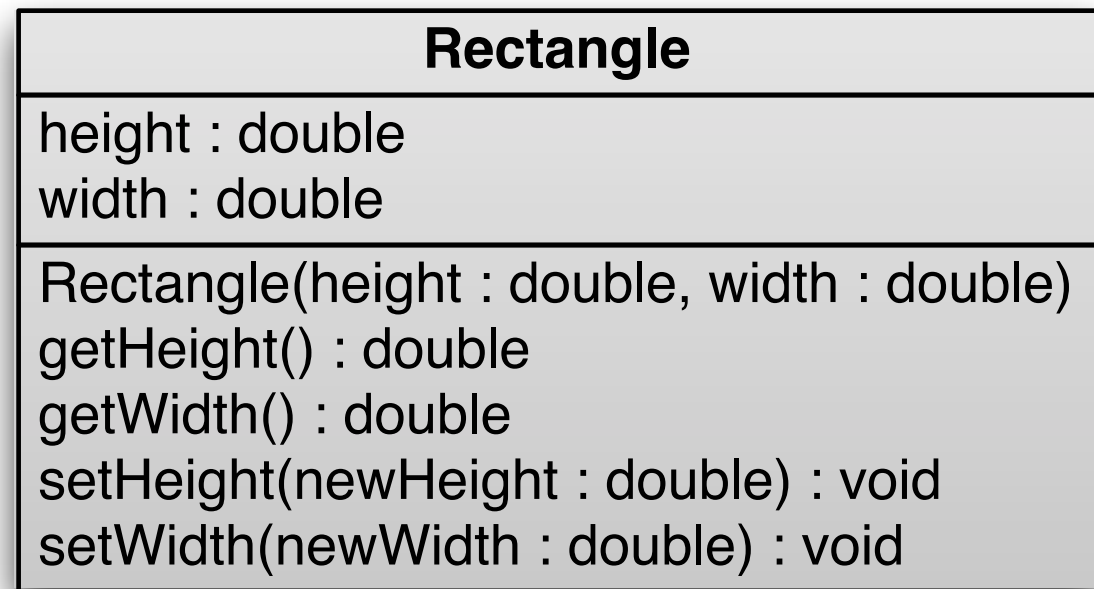
setHeight() and
setWidth() are
also inherited
from **Rectangle**

Square s = new Square(33);

s.setHeight(44);

System.out.println(s.getHeight()); // prints 44

System.out.println(s.getWidth()); // prints 33



Oops!

the **Square**

became

“rectangle-ish”

```
Square s = new Square(33);
```

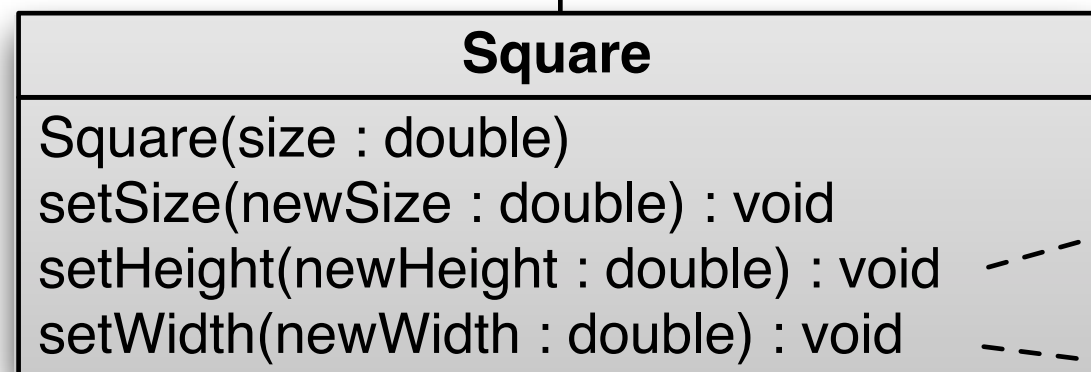
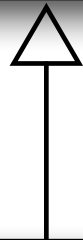
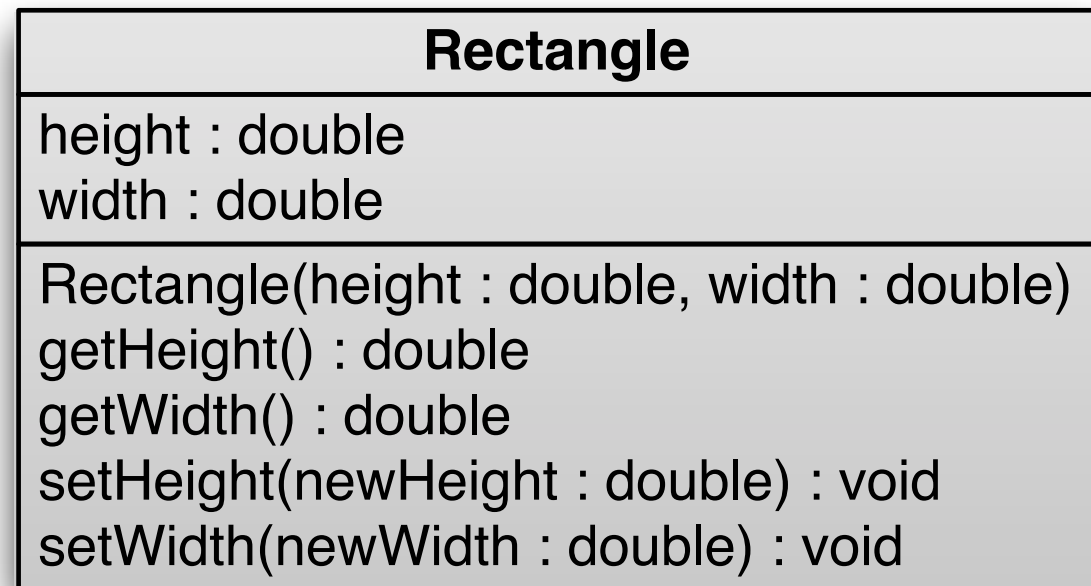
```
s.setHeight(44);
```

```
System.out.println(s.getHeight());
```

```
System.out.println(s.getWidth());
```

// prints 44

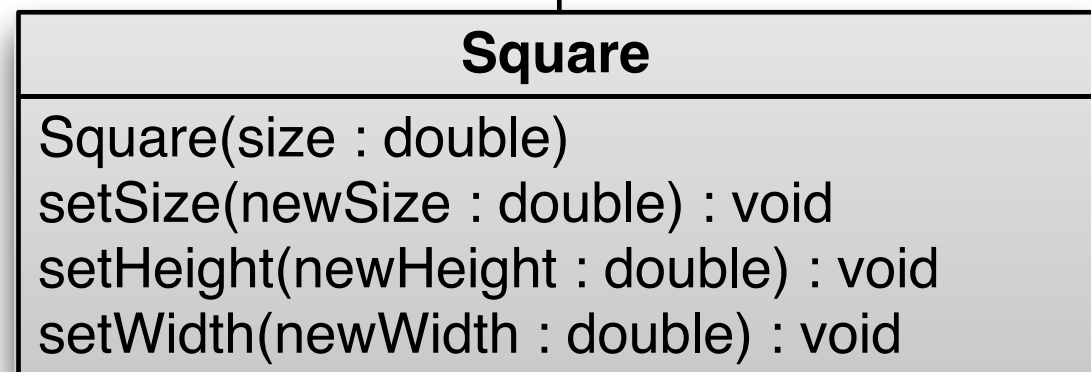
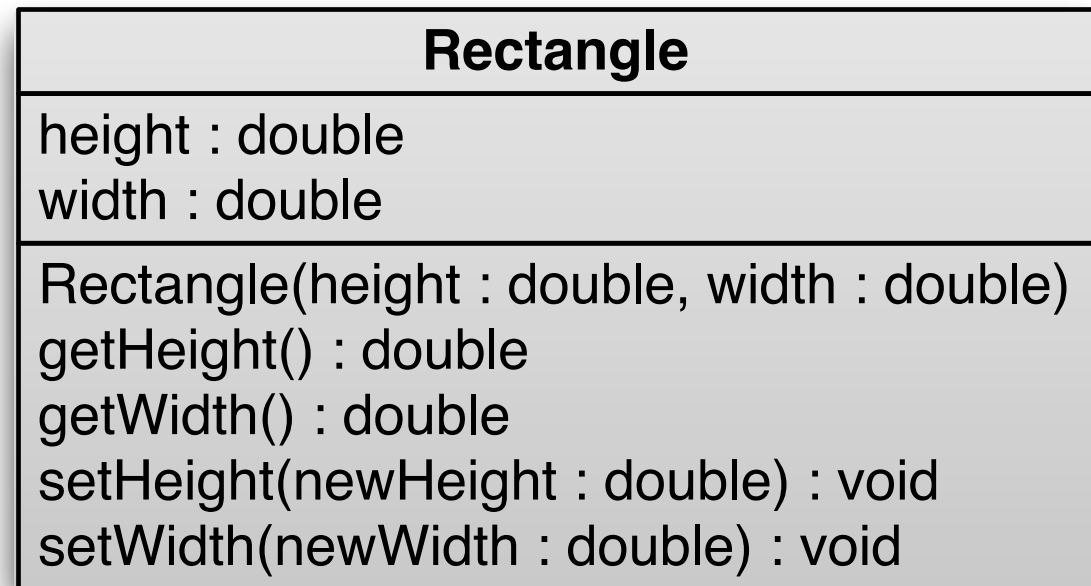
// prints 33



A **Fix?**

```
void setHeight(double size) {  
    super.setHeight(size);  
    setWidth(size);  
}
```

```
void setWidth(double size) {  
    setHeight(size);  
    super.setWidth(size);  
}
```



```
Square s = new Square(33);
```

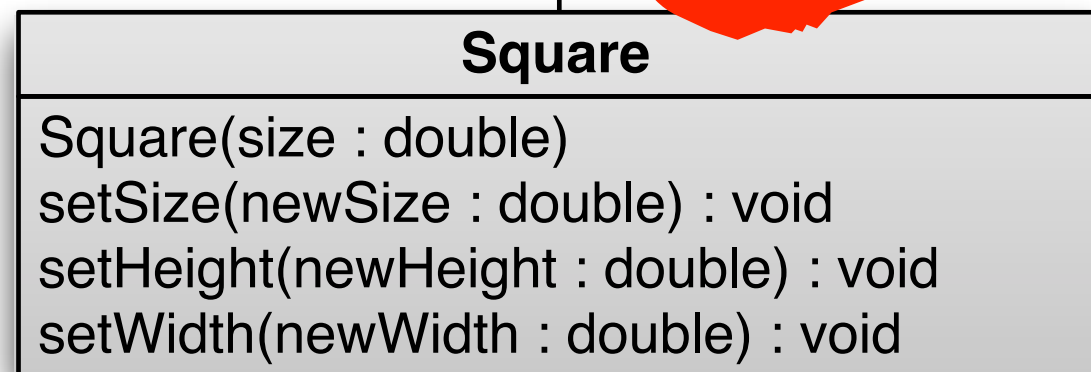
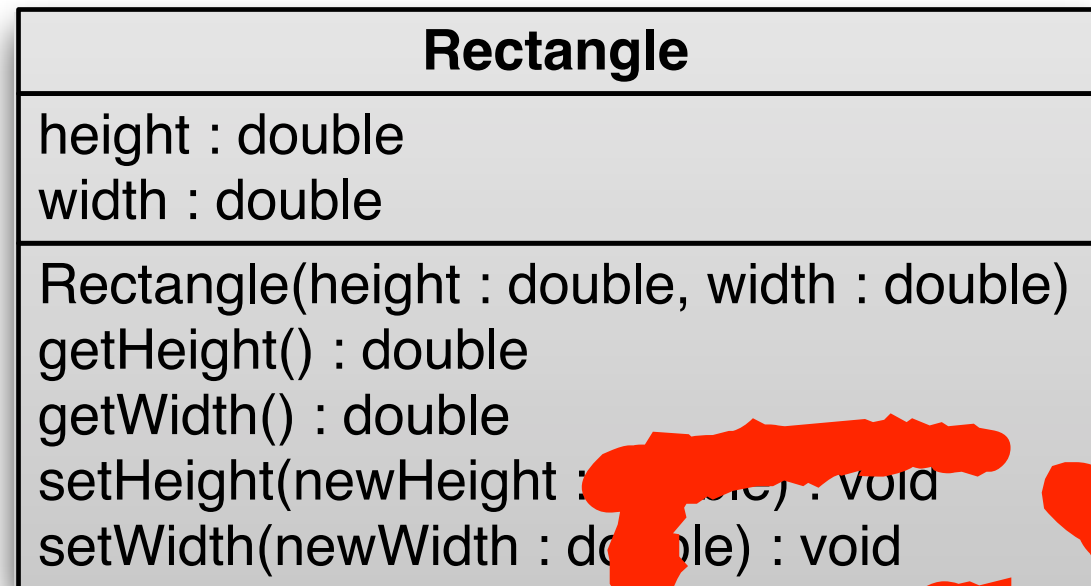
```
s.setHeight(44);
```

```
System.out.println(s.getHeight());    // prints 44
```

```
System.out.println(s.getWidth());    // prints 44 now, too
```

silently changes

width as well ...



evil!

silently changes

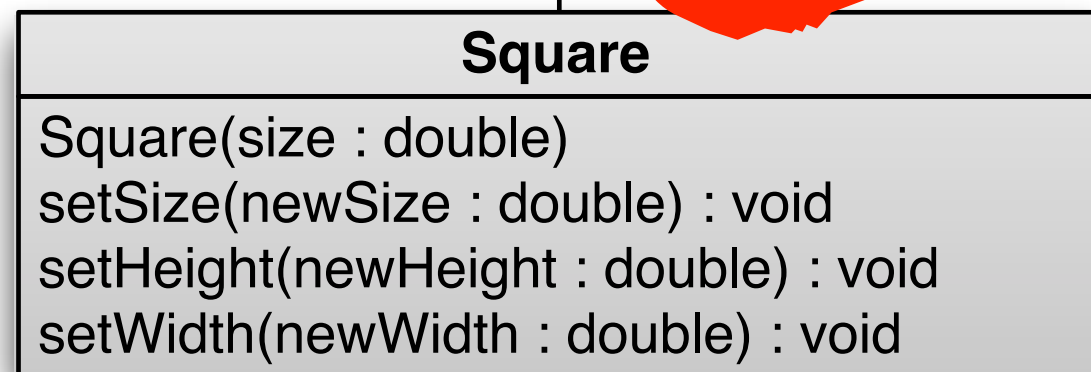
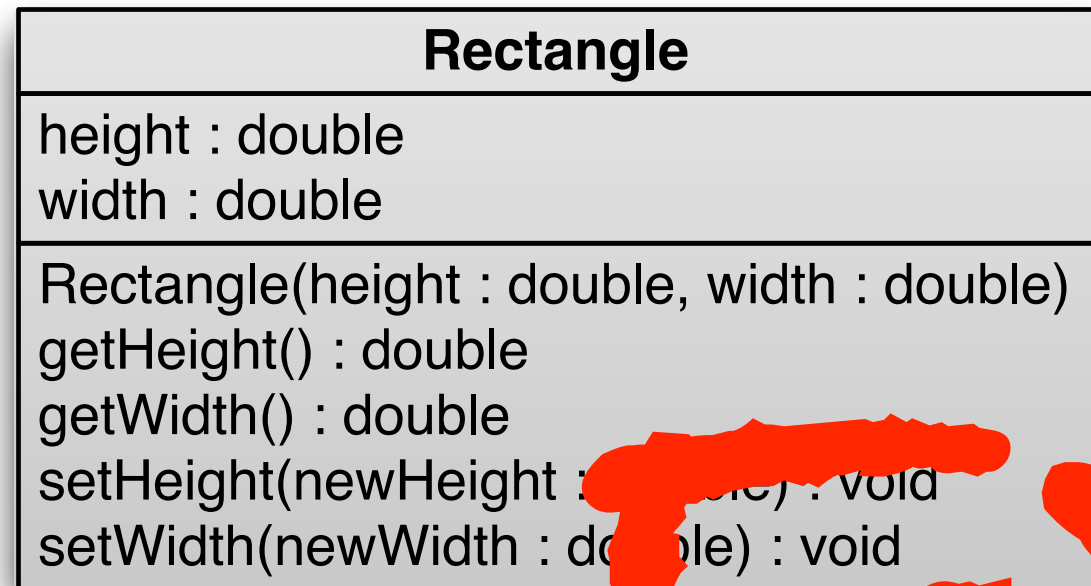
width as well ...

```
Square s = new Square(33);
```

```
s.setHeight(44);
```

```
System.out.println(s.getHeight()); // prints 44
```

```
System.out.println(s.getWidth()); // prints 44 now, too
```



ERROR!
assertion will

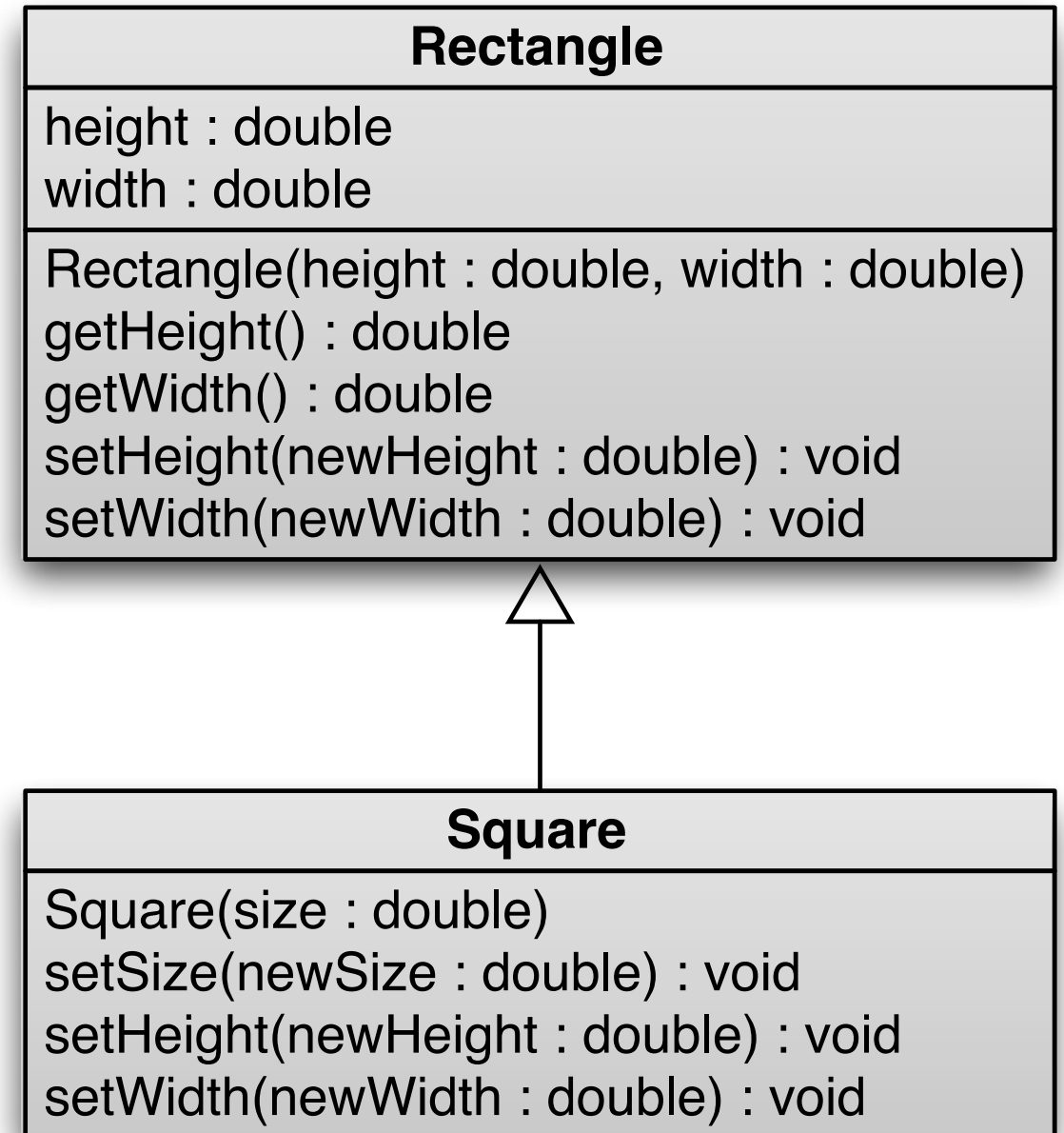
unexpectedly fail

```
void f(Rectangle r) {  
    r.setHeight(2); r.setWidth(3);  
    assert r.getHeight() * r.getWidth() == 6;  
}
```

f(new Square(1)); // can use Square where Rectangle is expected

Source of Problem

expected behavior that
getHeight() and
getWidth()
independently change
a single value does
not hold for Square

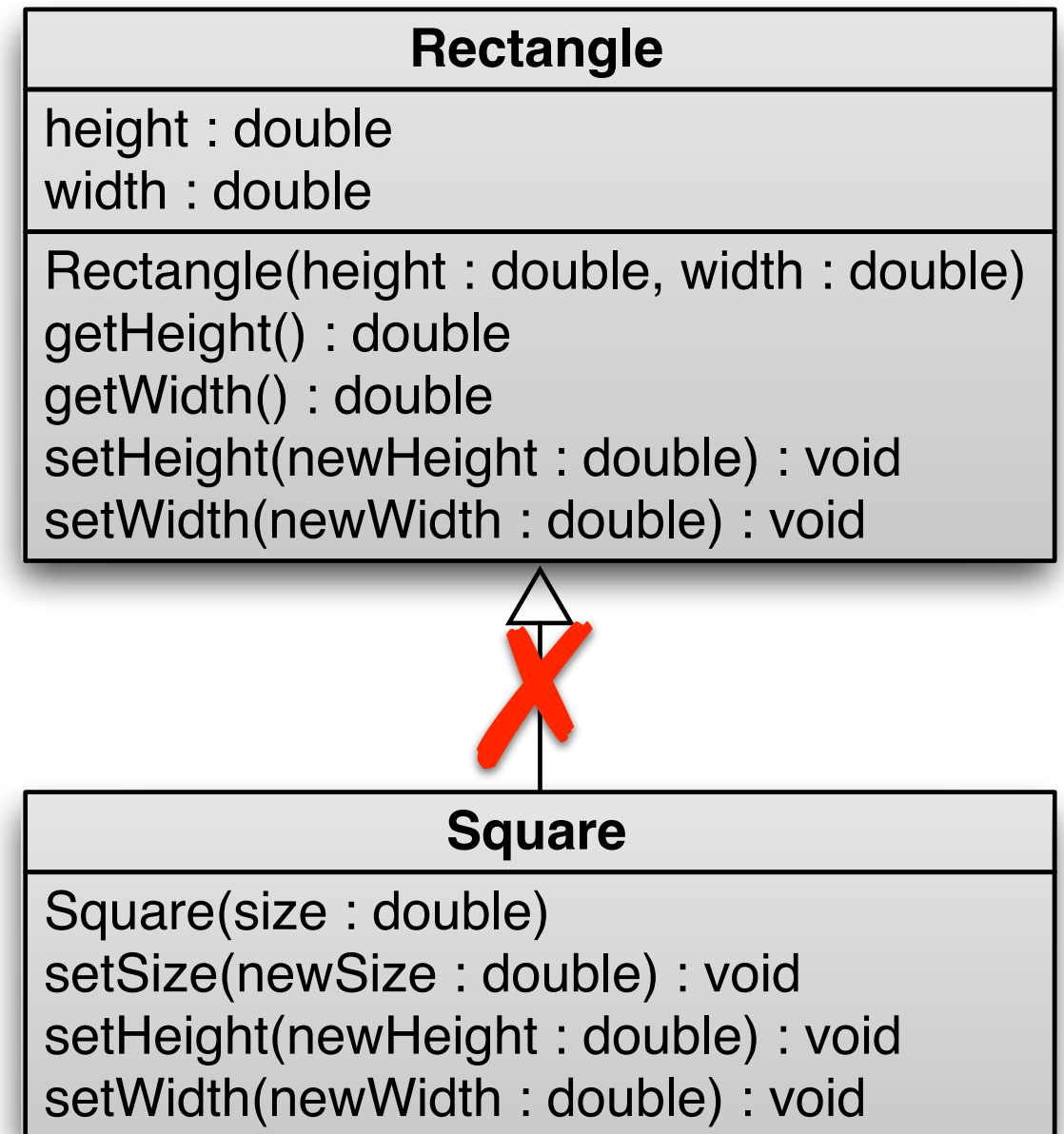


Implication

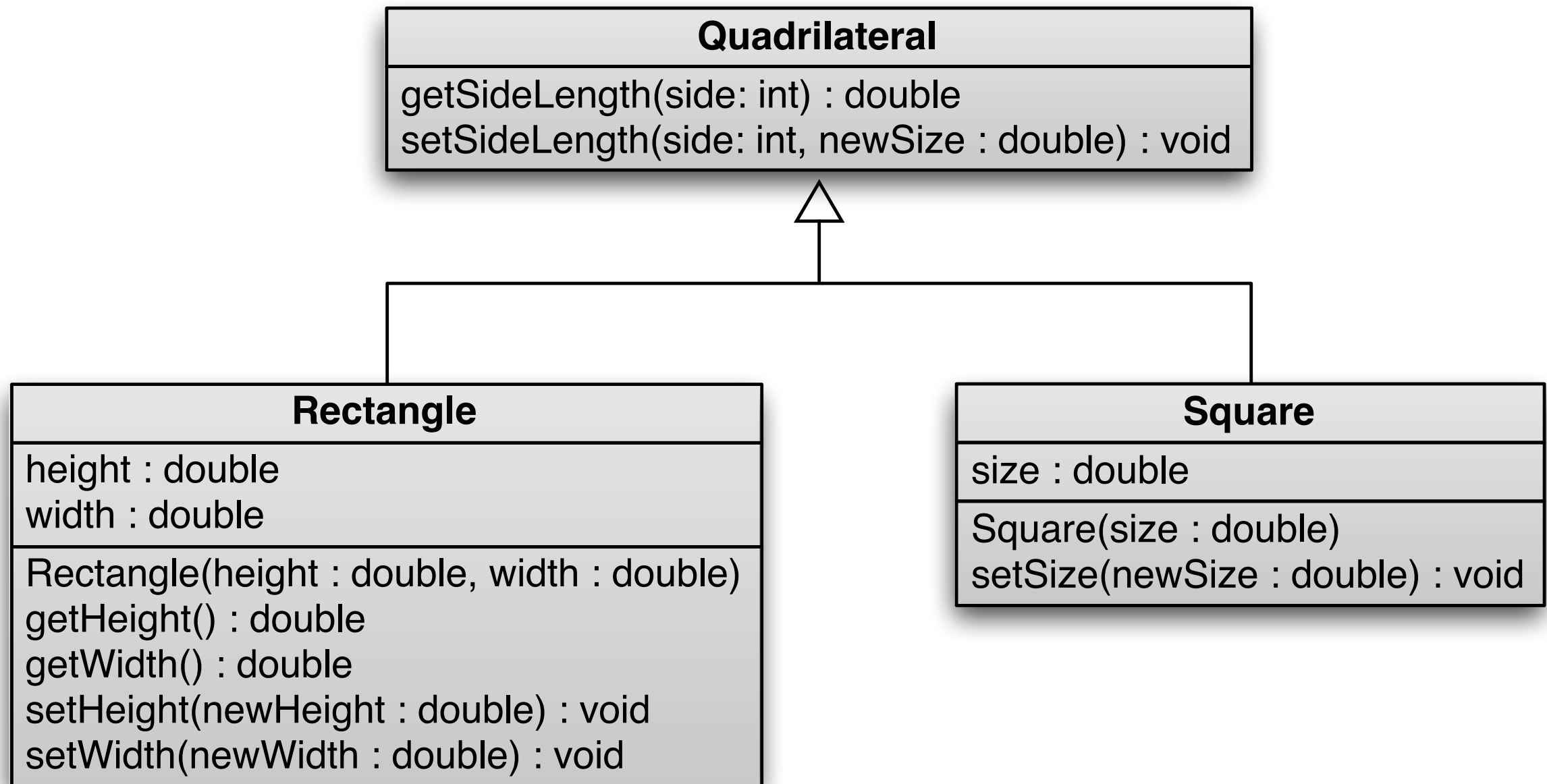
The Liskov Substitution Principle *does not hold* for Square and Rectangle—in other words,

Square is-a Rectangle is **wrong** from a **behavioral perspective**.

Thus, there must be **no inheritance** relationship between Square and Rectangle.



Potential **Approach**



attribution of **properties** now
more natural as well ...

